

Introduction, Tokens & Document Similarity

Language follows rules, but it is **more** than just these rules, it uses prior knowledge and reasoning. Context can in many cases change the meaning of the same sentences. Text input can refer to documents, social media posts, voice commands, search queries, web pages, medical records, books, etc.

Some example applications using text as input and output are voice assistants(siri, google, alexa, ...), Machine translation, Email suggestions/autocomplete. Can do anything from simply turning numbers into text form, to creative writing, e.g. writing a book.

Quickly advancing field - llms, deep learning, new & more powerful cpus and gpus.

1 Gen AI Developments

New language systems are essentially just trained by asking them to complete a sentence. the ELMo and BERT deep learning approaches developed a new model for DL that could succeed at different problems. More recently, GPT-4's deep learning model showed incredible new abilities - built on this was Github Copilot which can work as well with code as with written language.

Models getting bigger, increasing performance, but also **cost**. Most systems may be very good at a specific task, but research of generalisation is ongoing. Models that appear to generalise are often already trained on the task.

2 Document Similarity

If two pieces of text have overlap, we hypothesise that they are talking about a similar subject - the more they overlap, the higher the similarity. Text documents often contain a lot of formatting, links, and other content that need to be removed/reformatted depending on what you need, for example bold, italics, urls, tables, etc...

2.1 Measuring Overlap Between Documents/Posts

How should we measure overlap between two documents?
with individual letters? sequences of letters? Words?

Individual Letters Computers represent text as raw numbers for each character, do posts with similar underlying data have similar meaning?

I	R	N		B	R	U
73	82	78	32	66	82	85

Not really - the order and grouping is key - After all, "forty five" and "over fifty" have the same letters but different meanings.

Using individual letters can help detect other languages. Different languages often use different letters at different frequencies. This property can be used to detect other languages (though there are better ways). Frequencies of pairs/triples of letters can provide more data.

Sequences of Letters Bi-grams (n=2) are pairs of neighbouring characters, similarly tri-grams are triples of neighbouring characters. You can extract n-grams by running a sliding window across the text

[illegible]

This is more specific than individual letters - tri-grams can capture short words, and is easy to implement. However using small n wont be specific enough for many words, and large n may create n-grams that are very rare, and the same n-gram can come from words with different meanings ("ong" from "wrong" or "strong").

Words (Tokenization) Given a section of text, how do we split it up into "words"? - Tokenization

- Splitting a section of text into individual tokens
- A token or term is the technical term for a meaningful sequence of characters
- A token isn't exactly a word
- other languages have very different tokenisation challenges

Example sentence with some handmade rules: "John's father didn't have £100."

Idea 1: Splitting with whitespace (spaces, tabs, etc)

John's	father	didn't	have	£100.
--------	--------	--------	------	-------

Idea 2: Splitting with whitespace + punctuation (.,!@#\$%, etc)

John	'	s	father	didn	'	t	have	£	100	.
------	---	---	--------	------	---	---	------	---	-----	---

Ideal Tokenization: Requires some handcrafted rules to get:

John	's	father	did	n't	have	£	100	.
------	----	--------	-----	-----	------	---	-----	---

2.1.1 What is our Post About?

Some words are more important for representing the subject of the post. Some common words do not convey much meaning by themselves.

Anyone tried [Irn Bru](#)?

It's a **Scottish drink** and it's **banned** some **countries** and I was **wondering** if anyone here has **tried** it. It has quite a **unique taste** and it's not something you'd **forget quickly**. You either **love** it or **hate** it I think.

Anyone tried Im Bru?

It's a Scottish drink and it's banned some countries and I was wondering if anyone here has tried it. It has quite a unique taste and it's not something you'd forget quickly. You either love it or hate it I think.

A common practice is to remove words with minimal meaning - known as stop-words.

3 Stemming

Stems: The core meaning bearing units, for verbs, think of the infinitive (for example, to love).

Affixes: Bits and pieces that adhere to stems. Often grammatical additions for conjugation or tense.

Stems and affixes are broadly known as **morphemes** - small meaningful units that make up words.

3.1 Stemming

is the process for reducing inflected words to their stem or root form, often is rule-based and removes common suffixes. Generally works on single words with no context. Can make mistakes but often they aren't that bad.

The Porter Stemmer is a popular stemming algorithm, working as follows

Step 1a		Step 2 (for long stems)
sses → ss	caresses → caress	ational → ate relational → relate
ies → i	ponies → poni	izer → ize digitizer → digitize
ss → ss	caress → caress	ator → ate operator → operate
s → ∅	cats → cat	...
Step 1b		Step 3 (for longer stems)
(*v*)ing → ∅	walking → walk	al → ∅ revival → reviv
	sing → sing	able → ∅ adjustable → adjust
(*v*)ed → ∅	plastered → plaster	ate → ∅ activate → activ
...		

Stemming ignores meaning, two words with different meanings can end up with the same stem, for example emergency and emerge both have the stem "emerg"

3.2 Lemmatization

Lemmatization is stemming + a bit of meaning - it is slightly smarter than basic stemming, it may use the part of speech (e.g. verb/noun) to get some sense of meaning and apply different rules. for example, emergency and emerge to not get broken down.

Ideally two words have the same lemma if they have the same stem, same part of speech, and have essentially the same meaning.

3.3 Importance of Context

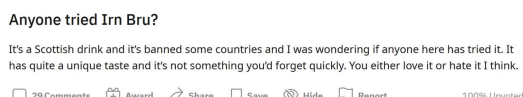
Words don't work independently and neighbouring words can change their meaning, how can we capture this?

Word n-grams can capture additional meaning. You can go beyond individual words into pairs or triples of words. Word bi-grams could capture "irn bru" as two words instead of "irn" and "bru" individually.

Word bi-grams and tri-grams are fairly powerful but can be computationally expensive

4 Set Similarity

Can we find other reddit posts that are similar to the one below?

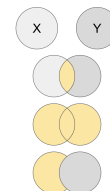


We can compare documents with sets of tokens. A set is a group of unique items, a document can be represented as a set of its tokens.

A set has no concept of item frequency, a word that occurs many times is treated the same as one that appears once. Most languages have sets as a standard data structure, and computation when comparing sets is very efficient.

4.1 Set Operations

- $|X|$ = number of elements in a set X
- $X \cap Y$ = intersection of sets X and Y
- $X \cup Y$ = union of sets X and Y
- $X - Y$ = set difference of X and Y



4.2 Similarity Measures

$\text{Sim}(X, Y)$ is a function that calculates a similarity measure where X and Y are the sets of tokens for two documents to be compared. $\text{Sim}(X, Y)$ should be high if they are similar and low if they are dissimilar.

4.2.1 Number of Overlapping Words

$$\text{sim}(X, Y) = |X \cap Y|$$

Where X and Y are sets of words for two documents, $\cap$ is in the intersection and $|Z|$ is the count of elements in a set Z . Can argue that two documents are similar if they contain many of the same tokens. But, need to be normalised, a long document that contains a large selection of words would match with lots of documents so need to factor in the number of tokens in each set.

4.2.2 Overlap Coefficient

$$\text{sim}(X, Y) = \frac{|X \cap Y|}{\min(|X|, |Y|)}$$

Where X and Y are the sets of words, $\cap$ is in the intersection and $|.|$ is the count of elements in the set. This essentially is the % of unique tokens in the smaller document that appears in the larger document

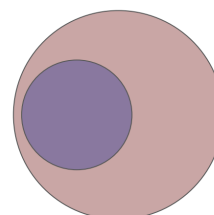
The improvements using this method are that now we are using the number of unique tokens in X and Y to normalise the measure, bounded between 0 and 1, where 0 means no overlap, and 1 meaning that one set is a subset of another.

4.2.3 Sørensen-Dice Coefficient

$$\text{sim}(X, Y) = \frac{2|X \cap Y|}{|X| + |Y|}$$

Properties:

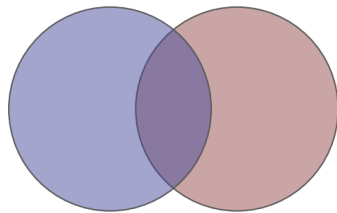
- bounded between 0 and 1
- 0 means no overlap and 1 is perfect overlap



Difference with overlap coefficient: Will only be 1 if the two sets are exactly matching, not if one is a subset of the other

4.2.4 Jaccard Similarity

$$\text{sim}(X, Y) = \frac{|X \cap Y|}{|X \cup Y|}$$



The number of overlapping tokens divided by the number of unique tokens across the two documents. It is again bounded between 0 and 1. its differences from Sorensen-Dice coefficient

Jaccard distance satisfies the triangle inequality such that $\text{dist}(X, Z) \leq \text{dist}(X, Y) + \text{dist}(Y, Z)$. This makes Jaccard a metric

Sorensen-Dice does not satisfy this - making it a semi metric

The following are properties of a metric:

- Distance between X and X is zero: $d(X, X) = 0$
- Positive: $d(X, Y) > 0$ where $X \neq Y$
- Symmetric: $d(X, Y) = d(Y, X)$
- Triangle Inequality holds: $d(X, Z) \leq d(X, Y) + d(Y, Z)$

Jaccard Example X : Unique tokens in first document - "irn", "bru", "rocks", "i", "drink" "daily"

Y : Unique tokens in second document - "irn", "bru", "is", "bad", "sprite", "is", "better"

Jaccard Similarity:

$$\frac{|X \cap Y|}{|X \cup Y|} = \frac{|\{"irn", "bru"\}|}{|\{"irn", "bru", "rocks", "i", "drink", "daily", "is", "bad", "sprite", "better"\}|} = \frac{2}{10} = \frac{1}{5}$$

Geometric similarity, Text Distributions, and Clustering

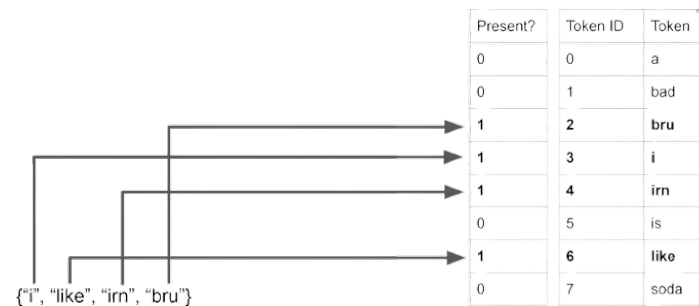
1 Sets to Sparse Vectors

Step 1: Build a vocabulary from all the tokens in the corpus, which is a collection of documents. Each token is assigned an integer ID.

Token ID	Token
0	a
1	bad
2	bru
3	i
4	irn
5	is
6	like
7	soda
8	taste
9	the

Step 2: Build a vector where the indices of the words in the set are all to 1 (all others 0). This is called boolean or

one-hot encoding. The length of the vector is the vocabulary size.



Present?	Token ID	Token
0	0	a
0	1	bad
1	2	bru
1	3	i
1	4	irn
0	5	is
1	6	like
0	7	soda
0	8	taste
0	9	the

In practice, most of the values in this vector will be 0. Vocabulary size » document length, since most value are 0, we call this data **sparse**. Lets say we allocate an entire vector for this data (e.g. as an array): {"i", "like", "irn", "bru"} = {0,0,1,1,1,0,1,0,0,0,...}. The drawback of this is we have lots of wasted memory storing 0s.

An often better approach is to store as a sparse representation, where we store Token ID - Value pairs which are non-zeros. {"i", "like", "irn", "bru"} = [(2,1), (3,1), (4,1), (6,1)] For now, value is always 1.

1.1 Sparse Data vs Sparse Representation

Your data (e.g. document vector) can be either sparse or dense, you can choose to represent this data in various ways.

	Dense (mostly non-zero)	Sparse (mostly zero)
Dense (fixed-array length)	Good idea	
Sparse (pairs of ID/Values)		Good idea

1.2 UNK Tokens

With Bag-of-Words, you must define your vocabulary - what happens when you see a new token? Use an *unknown* token, known as UNK.



Token ID	Token
1	bru
2	i
3	irn
4	like
5	UNK

2 Beyond One-Hot Encoding

2.1 Bag-of-Words Model - Term Frequency

Assumption: If a term occurs frequently in a document it should imply something about what the document is about. This is a relaxation of the binary occurrent assumption. Recording the term frequency information provides more information of "about-ness"

In memory this can be:

- Dense Rep: [0,0,2,1,2,0,2,0,1,0,...]
- Sparse Rep: [(2,2), (3,1), (4,2), (6,2), (8,1)]

2.2 Problems with Term Frequency

Even though important tokens are usually common in a document, common words are not necessarily important

Token	Frequency
the	14
to	4
in	4
djokovic	4
a	4
on	3
his	3
after	3
would	2
was	2
visa	2
tennis	2
start	2
open	2
of	2
novak	2
he	2
has	2
day	2

- Articles/Pronouns - Essentially all documents in English will use stop words, remember, we define a list of stop-words that should be removed
- Words that are frequent but do not convey much information, e.g. "BBC" in a corpus of news articles, "Research" in a corpus of scientific papers
- Co-relations between words, e.g. novak and djokovic occur much more frequently together than separately.
- Magnitudes of term frequencies, e.g. is an article about "djokovic"(4) twice as much as its about "visa"(2)

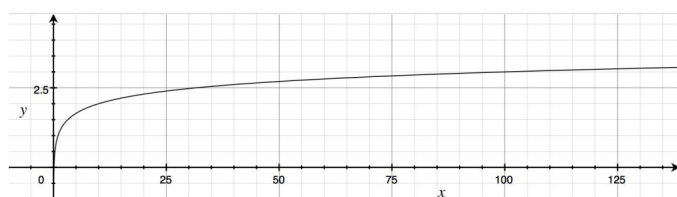
2.2.1 Problem 1: Raw Term Frequency (tf)

The term frequency ($tf_{t,d}$) of term t in document d is defined as the number of times that t occurs in d . Raw rf has a problem - it implies that a document with 10 occurrences of a term is 10 times more about the term than a document with 1 occurrence. **Aboutness does not increase linearly with term frequency.**

Solution: Sublinear TF Scaling The log frequency of term t in d is

$$w_{t,d} = \begin{cases} 1 + \log_{10}(tf_{t,d}), & \text{if } tf_{t,d} > 0 \\ 0, & \text{otherwise} \end{cases}$$

What impact does this have on very large term frequencies?



Example: $0 \rightarrow 0, 1 \rightarrow 1, 2 \rightarrow 1.3, 10 \rightarrow 2, 100 \rightarrow 4$

2.2.2 Problem 2: How 'Useful' is Each Term?

Words that occur in many documents do not provide much new information (BBC" in a corpus of news articles, "Research" in a corpus of scientific papers).

Idea: Measure how many documents a term appears in to predict how useful it is - common terms are not very helpful, and rare terms (misspellings, etc) are sometimes also not informative and sometimes removed entirely. This is called **Document Frequency** - How many **documents** in the collections contain a term?

However, terms that occur in more documents are **less** useful than rarer ones, a higher document frequency indicates a term is less important. Therefore, we want the inverse of of document frequency: A higher **Inverse Document Frequency (IDF)** indicates a term is **more** important

$$idf_t = \log\left(\frac{N}{df_t}\right)$$

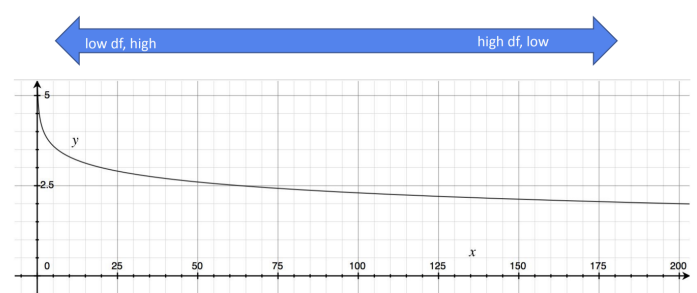
Note that we usually use $\log(N/df_t)$ instead of N/df_t to non-linearly dampen the effect of IDF, sometimes we don't specify a base for the log, in examples, we tend to use base 10 for convenience, but 2 is more common in practice.

Inverse Document Frequency (IDF) Suppose $N = 1$ million

term	df_t	idf_t
calpurnia	1	6
animal	100	5
sunday	1,000	4
fly	10,000	3
under	100,000	2
the	1,000,000	1

$idf_t = \log_{10}\left(\frac{N}{df_t}\right)$ There is one idf value for each term t in a collection.

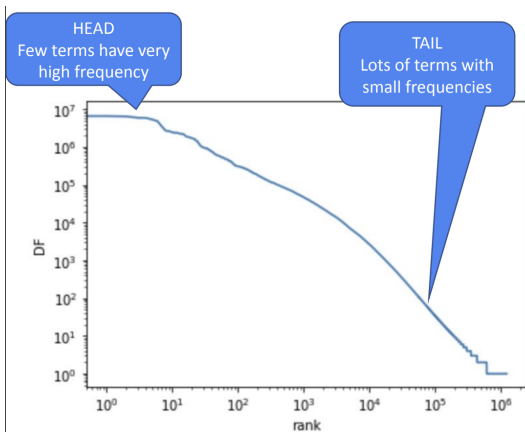
Graphically;



Ranking tokens by document frequency...

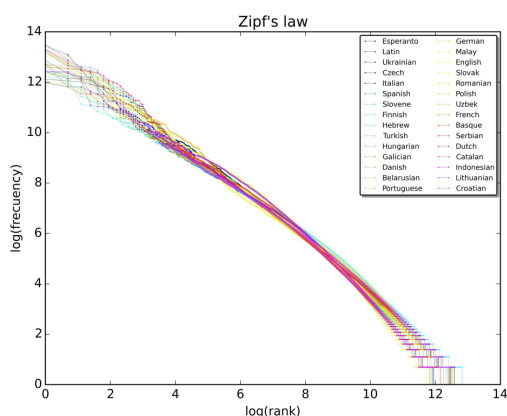
Rank	Token	DF
1	the	7,714,694
2	of	6562777
3	and	6432856
4	a	5919793
5	to	5847920
6	in	5321030
...		
1,242,320	tukana	1

On logarithmic scales,



The most frequent terms are not very useful, but there are only a few of them, for example, "the", "a", etc. There are lots of infrequent terms, but they can be useful. This is zipf's law - it shows us why IDF is useful and why we apply a logarithm to df.

2.3 Zipf's Law



A statistical property of language, this can be approximated as a straight line in a log-scale. Zipf's law is an example of a power law - many power law distributions occur in nature.

2.4 tf-idf Weighting

The tf-idf weight of a term is the product of its tf weights and its idf weight.

$$w_{t,d} = (1 + \log(tf_{t,d})) * \log\left(\frac{N}{df_t}\right)$$

This is a strong weighting scheme for text similarity, increases with the number of occurrences within a document, and increases with the rarity of the term in the collection. tf-idf weighting has many variants...

Term frequency		Document frequency	
n (natural)	$tf_{t,d}$	n (no)	1
l (logarithm)	$1 + \log(tf_{t,d})$	t (idf)	$\log \frac{N}{df_t}$
a (augmented)	$0.5 + \frac{0.5 \times tf_{t,d}}{\max_t(tf_{t,d})}$	p (prob idf)	$\max\{0, \log \frac{N-df_t}{df_t}\}$
b (boolean)	$\begin{cases} 1 & \text{if } tf_{t,d} > 0 \\ 0 & \text{otherwise} \end{cases}$		
L (log ave)	$\frac{1 + \log(tf_{t,d})}{1 + \log(\text{ave}_{t \in d}(tf_{t,d}))}$		

2.4.1 More Advanced Options

Document Length: Normalise to deal with longer versus shorter documents.

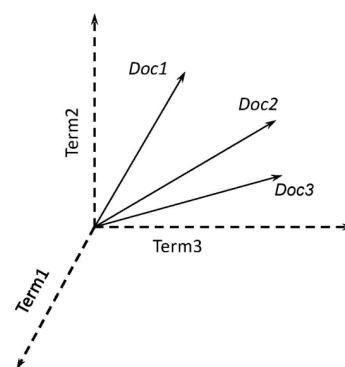
Replace Document Frequency: Use the times a word appears in many documents (collection frequency) instead of the number of documents.

Probabilistic Interpretation: Use the likelihood of words appearing (discussed later)

3 Geometric Similarity

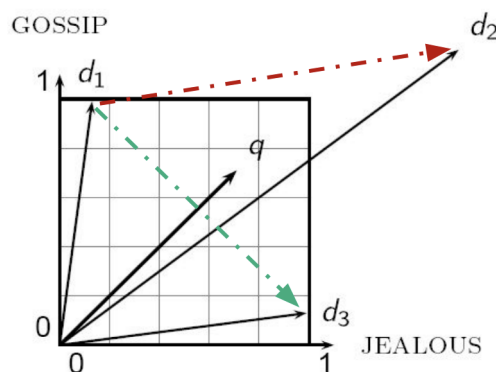
3.1 Vector Space Model

We can plot the vectors for each document.

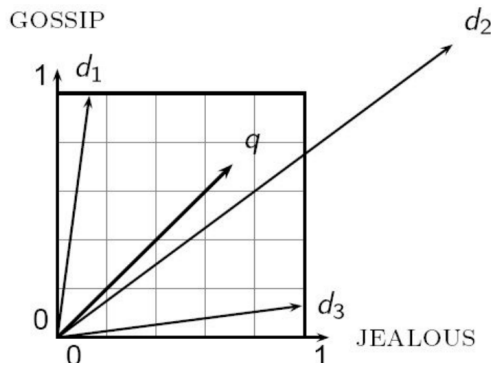


Assumption: Documents that are "close together" in vector space "talk about" the same things, i.e. are similar. 3D pictures are useful, but can be misleading for high-dimensional space.

To formalise vector space proximity, we could measure the distance between the endpoints of the vectors, this is called euclidean distance. $\|\vec{D}_1 - \vec{D}_2\|$. The problem is the distance is large for vectors of different lengths. The euclidean distance between D_1 and D_2 is larger than the distance between D_1 and D_3 , even though the distribution of terms in D_1 is more similar to D_3 than to D_2 .



Instead, we can use the angle (instead of the distance) between two vectors, the benefit of this is the vector magnitude is ignored so long/short documents can be similar, but we want high score for similar documents. Identical documents would have angle=0. We need a transform.



3.2 Vector Space Similarity

Similarity is the distance between points representing units of text. Similarity measure is more common than a distance or dissimilarity measure, e.g. Cosine correlation: $Cosine(D_1, D_2) = \frac{D_1 \cdot D_2}{|D_1||D_2|}$.

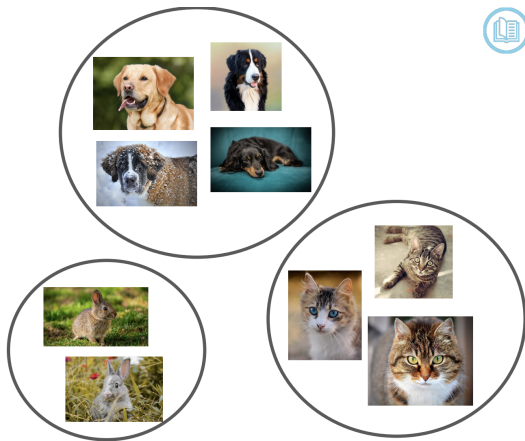
Dot product reminder We can do the dot product of two term vectors for D_1, D_2 . The dot product of two vectors is a scalar.

$$\text{dot product}(D_1, D_2) = \vec{D_1} * \vec{D_2} = \sum_{i=1}^{|v|} v_i w_i = v_1 * w_1 + v_2 * w_2 + ... v_{|v|} * w_{|V|}$$

where v_i is the count for word i in D_1 and w_i is the count for word i in D_2

4 Clustering

objective is to group a set of items by similarity. A classical problem in pattern recognition, statistics, and machine learning. When applied to documents or objects, they enable a wide range of tasks in text analysis applications.



An application of this could be customer segmentation, clustering customers by purchasing history, can then target specific clusters with tailored marketing

4.1 Background

The purpose of clustering is to discover the underlying structure of data by grouping similar objects together. in TaD the objects are typically text documents. Cluster documents by the similarity of the terms they contain. Documents within

the same cluster are assumed to be similar.

There is no definitive "right" number of clusters in a set, and the number as well as the clusters themselves will vary depending on the clustering algorithm used.

Clustering is typically an unsupervised learning task, unlike supervised classification, the clusters are not predefined. there is no labelled training data, and clusters are discovered by the algorithm.

4.1.1 Clustering Process

1. Derive a document representation, typically a vector of weighted terms.
2. Measure similarity between documents
3. Apply a clustering method
4. Check the validity/quality of the clustering.

4.2 Methods

there are some key properties of clustering algorithms to be considered:

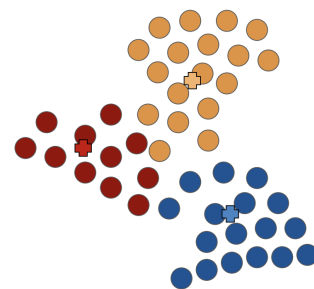
- Partitioning criteria - i.e. single level vs multi-level hierarchical partitioning, often multi-level hierarchical partitioning is desirable.
- Separation of clusters - Exclusive (e.g. one object belongs to only one cluster) vs overlapping (e.g. one object may belong to more than one cluster)
- Hard vs fuzzy - in fuzzy clustering, an object belongs to every cluster with some weight between 0 and 1, and weights must sum to one. Similar characteristics to probabilistic clustering.

4.2.1 Partitioning Clustering

Partitions a set of N documents into K disjoint clusters. Find the partition that optimises a specific criterion. Typically, a function of within cluster similarity and between cluster distance.

4.2.2 K-Means Clustering

One of the most popular clustering algorithms, takes a dataset and a k (number of clusters), and gives back the dataset partitioned into k clusters. Each cluster is represented by its centre point, known as the centroid.

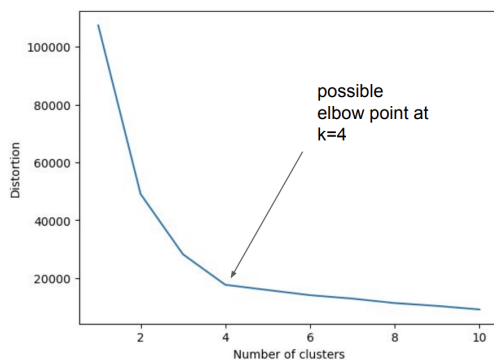


process The k-means algorithm selects k centroids, assigns each data point to its nearest centroid, and recalculates the centroids as the average of assigned points. The last 2 steps are repeated until convergence.

Pros and Cons K means tends to converge quickly, but it is sensitive to choice of initial seeds (may be local minima), however there are some clever methods for picking initial points. Its complexity is $O(N \cdot K)$ where n and k are the number of documents and clusters respectively.

Choosing Initial Centroid points Some classic approaches are **Forgy partition** - randomly selecting k data points from the dataset and using them as the initial centroids, or **Random partition** - assigning each point randomly to a cluster and calculating the centroids from each cluster.

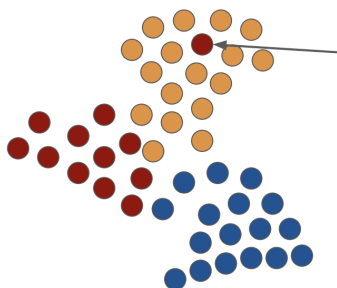
Choosing K Selecting k is both important yet tricky, as mentioned, k -means requires you to decide on the k and will give very different clusters for different values of k . Too small k could mean that important clusters are merged losing information, but too big k could make clusters with few data points and be too fine grained. With 2D and even 3D data, we can visualise and decide k visually, we need a general approach to choosing k .



The Elbow method of picking k Compute distortion = sum of squared distance of each point to its centroid. Graph distortion against the number of clusters. Look for a flattening of the curve (elbow point) - gives good choice of k , little gained with a value of k larger than this.

4.3 Measuring Clustering

There is no definitive measure for clustering, the silhouette coefficient does provide one metric to judge if individual data points have been put in the right cluster, here is an example of bad clustering



To measure the silhouette coefficient for a point i we use its

- **cohesion score** - average distance of i to the objects in the same cluster. - you want a small cohesion score.

- **separation score** - minimum of average distances of i to objects in another cluster

For an individual object, silhouette score is measured as

$$s(i) = 1 - a/b \quad \text{If } a < b$$

$$s(i) = 0 \quad \text{if } a = b$$

$$s = b/a - 1 \quad \text{if } a > b$$

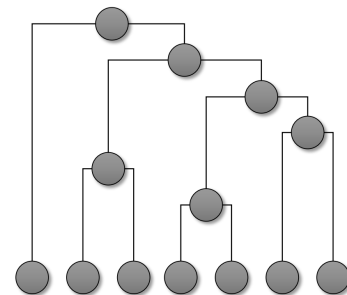
silhouette ranges from -1 to +1, A silhouette close to 1 implies an object is in the correct cluster, while a silhouette close to -1 implies the object is in the wrong cluster.

4.4 Web-Scale k-Means

there is a problem, running similarity over all items in the dataset is slow, especially for BIG datasets. We could instead of comparing against all objects, compare against a sample. The solution to this is mini-batch k -means, which uses mini batches to reduce computation time. Mini batches are random samples of the input data, dramatically reduces the conversion time, and only slightly worse than running full algorithm.

4.5 Hierarchical Clustering

This type of clustering results in a tree-like representation with clusters of highly similar documents nested within clusters of less similar objects. Can be agglomerative (bottom-up) or divisive (top-down).



K means is more efficient than hierarchical ($O(KN)$ rather than $O(N^3)$), and usually produces clusters of similar quality to hierarchical. K-means is the usual choice for an all-purpose clustering algorithm for a wide range of tasks, the main drawback being that k needs to be defined upfront.

4.6 Cluster Evaluation

4.6.1 Look at your data

Do a smell test, run basic checks for oddities (e.g. 90% of data in one cluster. Look at a few examples in each cluster. This method is not systematic.

4.6.2 Intrinsic

Internally consistent:

- Cohesion - items in a cluster should be similar to one another
- Separation - clusters should separate objects from one another
- Silhouette coefficient combines cohesion and separation.

Externally consistent:

- Judge whether pairs of objects are similar enough to be in the same group.

- Purity - measure the extent to which clusters contain a single class (required human labelled classes)

4.6.3 Extrinsic

Are you using the clustering as part of a larger task that can be measured? Cluster membership could be an input for another system, like a classifier.

Does this clustering improve the performance of the downstream task? could compare against random clustering or other methods

Language Modelling

language modelling is the task of building a predictive model of language. The goal is to predict the probability of the next word in a sequence and/or of a sentence of words.

1 Language modelling applications

Has many applications, including but not limited to:

- Speech recognition
- Optical character recognition
- Spelling correction
- Machine translation
- Authorship detection
- Auto-completion

1.1 Modelling Language

Historically, language was modelled with language statistics - a simple, surprisingly effective approach, and is still useful in many settings. More recently, using neural networks is the new approach - this is very expensive but highly effective. This is the technique between large language models.

2 Probability Review

2.1 Joint Probability

The chance of a set of events happening together, e.g. P(A drawn card is 8 and is red) = $2/52$

2.2 Conditional Probability

Chance of an event (A) conditioned on another event(B) - P(A given B)

$$P(\text{Card is 8} \mid \text{card is red}) = 2/26$$

$$P(\text{Card is red} \mid \text{card is 8}) = 2/4$$

2.3 Chain Rule

$$\begin{aligned} P(A, B, C, D) &= P(A|B, C, D) * P(B, C, D) \\ &= P(A|B, C, D) * P(B|C, D) * P(C, D) \\ &= P(A|B, C, D) * P(B|C, D) * P(C|D) * P(D) \end{aligned}$$

2.4 Independence

A and B are independent if $P(A|B) = P(A)$ (B has no influence on A), and if $P(B|A) = P(B)$ (A has no influence on B). And, $P(A, B) = P(A|B) * P(B) = P(A) * P(B)$ (using chain rule).

let A = card is 8 and B = card is red. $P(A, B) = 2/52$

$$P(A) * P(B) = 2/4 * 1/13 = 2/52$$

in this case, A and B are independent. Imagine the deck of cards has lost its 8 of clubs, this would mean that A and B are not independent anymore.

2.5 Probability Distribution

A probability distribution specifies the probabilities for all possible outcomes of an experiment (or set of experiments). Bernoulli/Binomial for experiments with only 2 outcomes, or can be Multinomial for >2 outcomes, e.g. total of multiple dice, or **probability of observing a word**

3 Language Modelling

A language model is something that specifies the following two quantities, for all words in the vocabulary.

1. Probability of a whole sentence or sequence : $P(w_1, w_2, \dots, w_3)$

2. Probability of the next word in a sequence : $P(w_{k+1} | w_1, \dots, w_k)$

$P(w_1, w_2, \dots, w_n)$ is short for $P(W_1 = w_1, W_2 = w_2, \dots, W_n = w_n)$ Multinomial random variable W_1 taking on value w_1 and so on. e.g., $P(\text{I, love, fish}) = P(W_1 = \text{I}, W_2 = \text{love}, W_3 = \text{fish})$

we model language by counting and normalising from source text, also known as a corpora.

3.1 Counting

One way to estimate the probability is the fraction of times you see it

$$P(w_1, w_2, \dots, w_n) = \text{Count}(w_1, w_2, \dots, w_n) / N$$

where count is the frequency of the sequence and N is the count of total number of n-length sequences.

3.1.1 Related Issues

Take for example the probability of the sequence P(I, like, chocolate, canada, and, basket, weaving). How often do people talk about their love of these three exact items? probably not much.

Conditioning on 4 or more tokens is hard because of sparsity, estimating probabilities from sparse observations is unreliable, and no solution for a new sequence.

We can use chain rule to decompose joint probability into a product of conditional probabilities.

$$\begin{aligned} P(\text{"The weather is snowy outside"}) &= P(\text{the}) \\ &\quad * P(\text{Weather} \mid \text{the}) \\ &\quad * P(\text{is} \mid \text{the weather}) \\ &\quad * P(\text{snowy} \mid \text{the weather is}) \\ &\quad * P(\text{Outside} \mid \text{the weather is snowy}) \end{aligned}$$

Estimating conditional probability with long contexts is difficult, and inefficient. We'll need to simplify the problem by making some assumptions

Markov Assumption

$$P(w_{k+1} | \underset{\text{-context window-}}{w_{i-k}, w_{i-k+1}, \dots, w_k})$$

Next event in a sequence depends only on its immediate past (context), instead of an arbitrarily long context, we rely on the last n words.

3.2 n-gram models

A model is often described by its order: the size of the context window $+1$ / ngram length.

Unigram	$P(w_{k+1})$
Bigram	$P(w_{k+1} w_k)$
Trigram	$P(w_{k+1} w_{k-1}, w_k)$
4-gram	$P(w_{k+1} w_{k-2}, w_{k-1}, w_k)$

Unigram Models Next event in a sequence is independent of the past

$$P(W) = \prod_i P(w_i)$$

An extreme assumption, but can be useful nonetheless. Used in a bag-of-words model and similarity. Not very good at modelling a sequence, nonsensical phrases or sentences can get high probability.

Bigram Models The next word is dependent on the previous word alone. This model is fast, efficient, and surprisingly effective

$$P(W) = \prod_i P(w_i | w_{i-1})$$

Example Given a sequence [the dog barks], what is the 'order' of this model, and what is the conditional probability of the last word: $P(\text{barks} | \text{the}, \text{dog})$.

"the dog barks" occurs 56 times in the collection, and "the dog" occurs 1190 times in the collection.

Trigram model

$$P(\text{barks} | \text{the}, \text{dog}) = 56 / 1190 = 0.047...$$

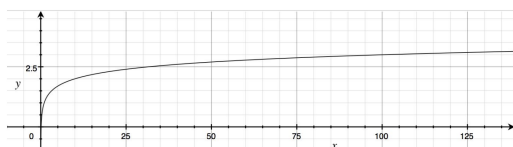
3.3 Practical LM Issues

Start & End Tokens We introduce special START and END tokens at the beginning of a sequence. This allows us to model the likelihood of beginning or ending of a sentence with a word (similar to html $\langle P \rangle$ and $\langle /P \rangle$).

Implicitly the first word of a bigram is $P(w | \langle \text{start} \rangle)$, sometimes $p(w)$. Allows identification of the end of a sequence, e.g. $P(\langle \text{end} \rangle | \text{"i like pie"})$

Log Space We do probability computations in log space for several reasons, being :

- To avoid underflow of very small numbers
- Adding is sometimes faster than multiplying
- Sometimes easier to interpret
- $\log(p_1 \times p_2 \dots \times p_n) = \log(p_1) + \log(p_2) + \dots \log(p_n)$
- The base of the log is arbitrary



3.4 Problem - Unique words

What about a sequence with a new word, for example, $P(\text{language models are fascinating, \#ilovetad})$. Model has never seen $\#ilovetad$ before, so $P(\text{string}) = 0$

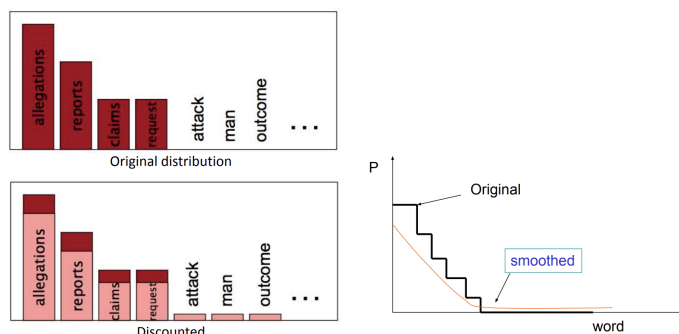
What do we do about never before observed words or sequences, and avoid assigning 0 probability to this event.

3.5 Smoothing

Goal of smoothing is to assign a low - but not 0 - probability to words or n-grams not observed in the text collection (training data). Text are just a sample from the language model. Missing words should not have zero probability of occurring. A non-occurring term is possible, even though it didn't occur in the document or training examples.

Smoothing is a technique for estimating probabilities for missing (or unseen) words. Lower the probability estimates for words that are seen in the document text, assign the "left over" probability to the estimates for the words that are not seen in the text.

3.5.1 Discounted Smoothing



Take some probability of the non-zero tokens and give it to the tokens with zero probability

3.5.2 Add-K / Laplace Smoothing

A type of discounted smoothing. Assume that there was some additional documents in the corpus, where every possible sequence of words was seen exactly k times.

$$P_{add-k}(t|\theta) = \frac{\text{count}(t, \theta) + k}{\sum_{t'} \text{count}(t', \theta) + k|V|}$$

where $\text{count}(t|\theta)$ is the count of term t in the corpus θ and $|V|$ is the number of unique words in the vocabulary. Add-one smoothing can be used with $k=1$, but often bad as too much probability is taken from seen events. K can be a fraction.

3.5.3 Interpolated Smoothing

Combines a higher order (more sparse) model with a lower order (less sparse) model, e.g. trigram, bigram, unigram. Example for a trigram model:

$$P_{interpolated}(t_i | t_{i-1}, t_{i-2}) = \lambda_1 P(t_i | t_{i-1}, t_{i-2}) + \lambda_2 P(t_i | t_{i-1}) + \lambda_3 P(t_i)$$

$$\lambda_1 \geq 0, \lambda_2 \geq 0, \lambda_3 \geq 0 \text{ and } \lambda_1 + \lambda_2 + \lambda_3 = 1$$

3.5.4 Jelinek-Mercer Smoothing

Interpolate between the frequency of a token in the document ($P(t|\theta_d)$) and the frequency of a token in the whole corpus ($P(t|\theta_c)$)

$$P_{\text{jelinek-merc}}(t|\theta_d, \theta_c) = \lambda P(t|\theta_d) + (1 - \lambda) \lambda P(t|\theta_c)$$

$P(t|\theta_d)$ is the probability based on token in **this document**, $\lambda P(t|\theta_d)$ is the probability based on tokens across the whole corpus.

3.5.5 Other Approaches

There are many other smoothing models for text, but many have complicated parameters that require tuning. When in doubt, start simple. 'Stupid' backoff used at google.

- Computing large n-gram probabilities (up to length 5)
- If $P() > 0$ then use it, if 0 use $\alpha=0.4$ * next lower order model.

For example, if trigram is 0, use 0.4 * bigram probability

4 Evaluation

Unsurprisingly, a good language model should model the language well. Well formed sentences should be more probable. It should match the **grammatical** and **syntactic** rules of the language, e.g. $P(\text{"like i tad"}) < P(\text{"i like tad"})$, and **semantically**, should convey actual meaning, e.g. $P(\text{"i saw a supernova at the planetarium"}) > P(\text{"i saw an elephant at the planetarium"})$

Not all syntactically-sound sentences make sense, e.g. "colourless green ideas sleep furiously"

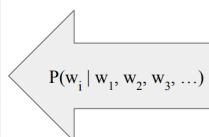
A trigram model captures more context than a bigram model, so it should be a "better" model - how do we measure this?

- **Extrinsic** - Measure performance on downstream application, e.g. spelling correction, speech recognition, translation, etc.
- **Intrinsic** - Design a measure inherent to the current task, challenging for language modelling, e.g. what does your language model make of "real text"

4.1 Calculating Probability of "real unseen text"

A good language model should give a higher probability for plausible/real text, e.g. the works of Charles Dickens.

Token	Probability of token from LM
It	0.6
was	0.3
the	0.2
best	0.8
of	0.3
times	0.1



Probability of text = 0.0008532, **probability of entire sequence** is calculated with $\prod P(w_i|w_1, w_2, w_3, \dots)$. The probability gets very small very quickly. To deal with the tiny probabilities of a whole sequence,

we work with per-token probabilities, for a sequence with N tokens, per-token $P = N\text{th root of } P(\text{sequence}) = P(\text{sequence})^{1/N}$. This is known as the **geometric mean** of the probabilities

For the example above, this gives us a per-token probability of 0.309. Next we need the perplexity, calculated as $1/\text{per-token probability}$.

$$\text{perplexity} = \frac{1}{\sqrt[n]{\prod P(w_i|w_1, w_2, w_3, \dots)}}$$

where n is the length of the sequence. For the earlier example, perplexity is 3.240.

4.2 Perplexity

Low perplexity means the language model gave high probabilities for the text, so wasn't surprised by it.

High perplexity means the language model gave low probabilities for the text, so was surprised by it.

Perplexity ranges from 1 to the size of the vocabulary

Perplexity of a random language model What if each token was just randomly picked from a vocabulary of 10000 words? If each probability was 0.0001, this gives the probability of text $1e-24$, per word probability of 0.0001, and 10000 perplexity.

Perplexity of a perfect language model If each token was guessed perfectly, the probability of text, per-word probability, and perplexity is all 1.

4.2.1 Practically Calculating Perplexity

Multiplying probabilities directly is a bad idea, number get tiny and hard to compute. We work with log-transformed probabilities instead - multiplication becomes addition.

How does this affect calculating perplexity - log transformed per-word probability gives an estimate for cross-entropy. This has direct links to perplexity.

0.032473218972138
 x 0.321684321351642
 x 0.987312654123324
 x 0.426545433489721
 x 0.897324231982315
 x 0.165643216843212
 x 0.567423136542131
 x 0.879238602321546
 x 0.065432852138621
 x 0.168412805642134
 x 0.789123681241238
 x 0.213432165423184
 x 0.572618465132987
 x 0.546723135448973
 x 0.370917652048341
 x 0.232654853282831
 x 0.218281238386153
 x 0.024272176096324
 x 0.245823244659824

4.2.2 Cross-Entropy

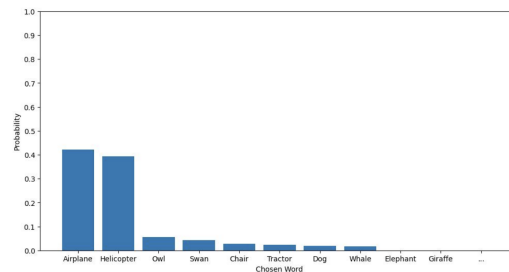
A measure of the difference between the true probability distribution and the language model's probability distribution. We treat some "real" text as a sample from the true probability distribution. In reality, we should use lots of text so our cross-entropy estimate is more accurate.

Average log probability	-1.696
Cross-entropy	1.696

Token	Probability of token from LM	Log2 probability
It	0.6	-0.73697
was	0.3	-1.73697
the	0.2	-2.32193
best	0.8	-0.32193
of	0.3	-1.73697
times	0.1	-3.32193

Cross entropy equals the negative of the average log probability (base 2) of each word.
 $\text{perplexity} = 2^{\text{cross-entropy}}$

Token	Probability of token from LM	Log2 probability
It	0.6	-0.73697
was	0.3	-1.73697
the	0.2	-2.32193
best	0.8	-0.32193
of	0.3	-1.73697
times	0.1	-3.32193
Average log probability	-1.696	
Cross-entropy	1.696	
Perplexity	3.240	



Asking "is it an airplane or helicopter" first is the optimal strategy.

5.2 Entropy of a Probability Distribution

Entropy is the measurement of the average uncertainty of information in a probability distribution. Expected number of bits needed to encode a message. Informally, the total "surprise" - adding up the surprise of each possible event, weighted by its likelihood.

$$H(X) = - \sum_x P(x) \log_2(P(x))$$

Where $P(x)$ is the weighting by the likelihood, and $\log_2(P(x))$ is the number of bits to encode this event (the surprise). Let's calculate entropy for the elephant or giraffe question:

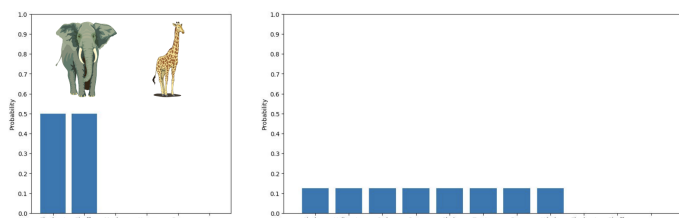
$$\begin{aligned} H(X) &= - \sum_x P(x) \log_2(P(x)) \\ &= -(0.5 \log_2(0.5) + 0.5 \log_2(0.5)) \\ &= -(-0.5 + -0.5) \\ &= 1 \end{aligned}$$

Hence, 1 question.

What about non-uniform distributions?

5 Link to Information Theory

The intrinsic measures - perplexity and cross-entropy - have their root in information theory. Most notably Claude Shannon worked on how to best encode information. His work led to the foundation of the information theory field.



Probability Distributions for which word is chosen
 the right requires 1 question - 1 bit of information content.
 The left requires 3 - 3 bits of information content.

5.1 Non-Uniform Distribution

If air plane and helicopter are more likely, asking for these would be better than to ask for something less likely.

x	P(x)	P(x)log ₂ (P(x))
Airplane	0.421	-0.525
Helicopter	0.393	-0.530
Owl	0.056	-0.233
Fly	0.042	-0.193
Swan	0.028	-0.145
Tractor	0.022	-0.123
Dog	0.020	-0.111
Whale	0.017	-0.099

$$\begin{aligned} H(X) &= - \sum_x P(x) \log_2(P(x)) \\ &= -(-1.959) \\ &= 1.959 \end{aligned}$$

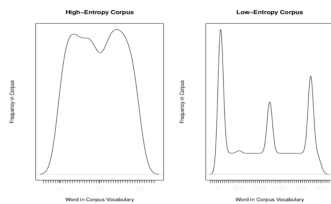
so around 2 questions, less than the 3 questions for uniform.

5.3 Entropy

Entropy is a key idea in data compression. Data with fewer possible values, or non-uniform distribution gives fewer bits to encode it in a compressed form.

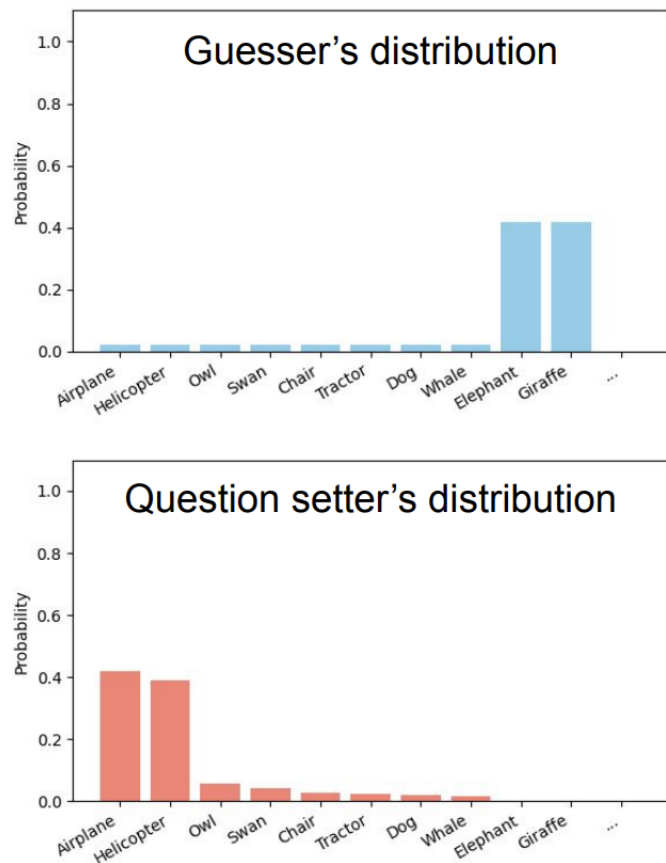
5.3.1 High vs Low Entropy for Language Models

A uniform distribution has high entropy - harder to predict a random draw from it. A peaked distribution has low entropy - easier to predict a random draw from it.



We know from Zipf's law that natural language corpora will usually look like this.

5.3.2 Cross Entropy



Guesser thinks setter will likely pick elephant or giraffe - **Model's distribution** setter change their strategy and is picking Airplane/Helicopter - **Observed distribution**. This is cross entropy - How many Yes/No questions to get the answer from the observed distribution given this assumption of the model's distribution.

First, measure between the two probability distribution the probability distribution from the LM and the observed distribution of words in our evaluation dataset. Its measured in bits.

This is related to perplexity, perplexity = reciprocal of our models probability of the observed data - Perplexity = $2^{\text{cross-entropy}}$

5.3.3 Comparing Language Models with Perplexity and Cross-Entropy

	Language Model A	Language Model B
Perplexity	32.7	16.4
Cross-Entropy	5.03	4.04

Figure 1: Example perplexities on a held-out text set

Perplexity often used to find a "better language model - this is specific to each dataset, and summarising a systems ability to model language by a single number is limiting.

Here, model B seems to perform better for this dataset, model A requires around 1 more bit to encode the observed data than model B.

6 Generating Text with a Language Model

Remember, we can find the probability of a word in a sequence using a language model: $P(w_{k+1}|w_1, w_2, \dots, w_k)$

This property allows us to envision what a likely continuation of the text would look like, e.g. you can take the maximum.

$\max_{x_1 \in V} P(x_1|w_1, w_2, \dots, w_k)$ "The dog _____ $\rightarrow$ barks"

$\max_{x_2 \in V} P(x_2|w_1, w_2, \dots, w_k, x_1)$ "The dog barks _____ $\rightarrow$ at"

$\max_{x_3 \in V} P(x_3|w_1, w_2, \dots, w_k, x_1, x_2)$ "The dog barks at _____ $\rightarrow$..."

This is a *greedy* generation strategy, the actual most likely sequence may be different. It's too expensive to explore the whole space.

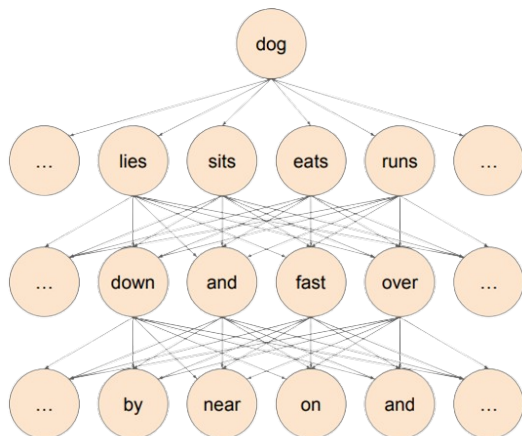
Always picking the most probable next token often creates boring, cyclical outputs, e.g. "the dog that the dog that the...". Instead, treat the probabilities as a distribution and sample from them. The token with highest probability is still the likeliest output, but other words also have a chance.

6.1 One Token at a Time

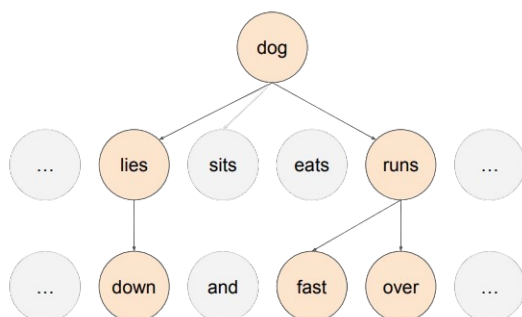
Generating one token at a time may not be ideal, for example, some tokens are rare and unlikely to be generated alone, but can be quite common and reasonable as part of a longer phrase. This can be common with names and phrases that start with uncommon words, e.g. "Barack Obama", "esoteric knowledge". We need generation to look a bit further ahead.

6.2 Beam Search

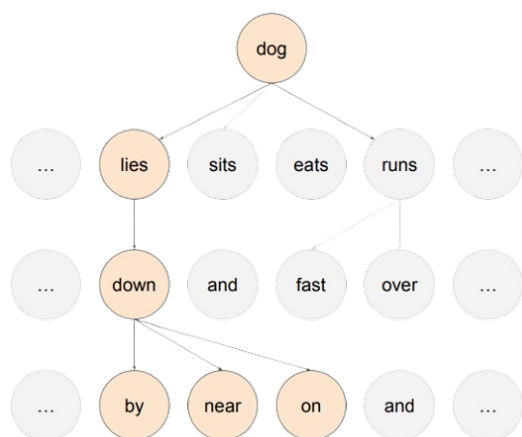
Cannot search all possible generations, even for a small number of tokens, to find likeliest. We can instead grow several candidate short sequences (token-by-token) and discard ones that become too unlikely. The beam width parameter decides how many to keep



For beam width = 3, we start by keeping the top 3 tokens for the first step and grow those paths. Next we add one extra token to the previous beams and keep the paths with the highest probability. In the following example, we discard the beams starting with "sits" as they had lower probability that "runs fast", "runs over", and "lies down".



We continue extending all of the previous paths and only keep the three most likely (for beam width=3). We discard some of the existing ones and find that the highest probabilities are with sequences starting with "lies down"



6.3 More Ideas for Generating Text

- Avoid extremely rare words - top k: Filter to only top k words before sampling. top p: Filter to words with top x% of probability, also known as nucleus sampling.
- Avoid repetition - Block repeated n-grams. Advanced contrastive search algorithm that blocks repeated text that looks similar to previous text.

Text Classification

Classification is the task of predicting which of a predefined set of classes (or categories) an object belongs to, in other words, assigning a class to an object based on some pre-trained knowledge.

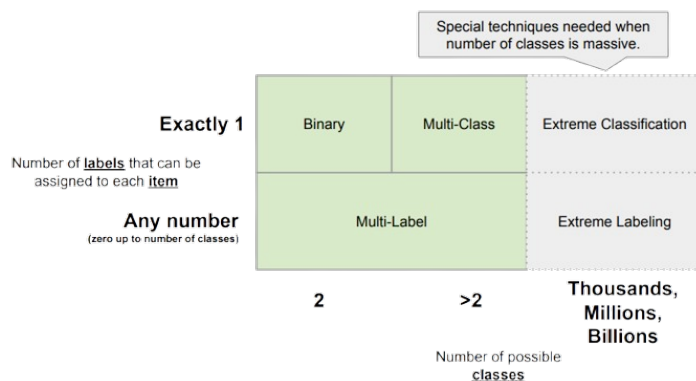
In our context, we would work with:

- Web pages
- Text documents, such as news articles
- Emails
- Product descriptions
- Natural language questions
- Social Media Posts
- Words themselves
- Really anything with words.

We use this to determine the subjects of articles to help users identify which users are interesting to them, and for spam classification so we can filter out comments we don't want to see. Text classification is an essential element in a wide range of modern tasks, including

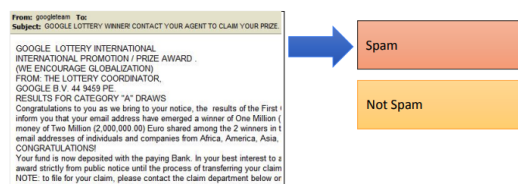
- **Topic Categorisation** - Is this document about technology, sports, lifestyle, etc.
- **Spam Detection** - Is this comment/email spam?
- **Sentiment Analysis** - Does this tweet express a negative opinion?
- **Language Identification** - Is this document written in Spanish?
- **Author Attribution** - Is this document written by John Smith?
- **Domain-Specific** - Is this content "clinical trial results" or "not clinical trial results"?
- **Discourse** - Is this reddit post a question, an answer, or clickbait?

1 Classes vs Labels



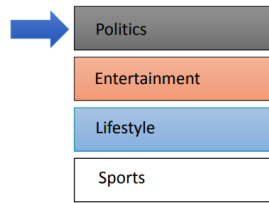
1.1 Binary Classification

Each item belongs to exactly one of two classes, for example, spam or not spam.



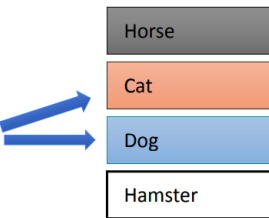
1.2 Multi Class

Each item belongs to 1 of >2 classes



1.3 Multi Label

Each item can belong to any number of classes.



2 Supervised Learning

Supervised learning has a right answer, and needs annotated label(s) for each sample. Unsupervised learning has no labels for data, and is used to explore data, finding patterns and groups - clustering is a popular unsupervised approach.

2.1 A Labelled Dataset

Data quality makes or breaks a machine learning project. Expensive to prepare as it typically involves human work, the labels must be assigned accurately and consistently for the classifier to learn a true representation of the task. Usually a good idea to have multiple assessors label the same items and measure inter-annotator agreement.

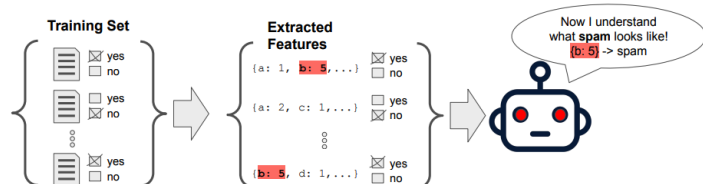
2.2 Representing Text with Numbers

Machine Learning algorithms work with numbers, representing both numerical and binary values. Often represented with multiple binary values, Nominal(categorical) and Ordinal(ordered categories) values. But how do we represent text?

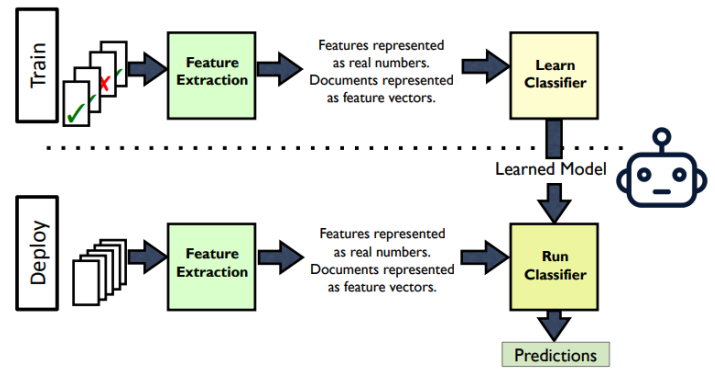
Vectorisation. we represent text as binary features of word occurrences (one-hot encoding) or as continuous features (TF and TF-IDF weights)

2.3 Training Process

Features from the input documents are extracted. The features and classes/labels from training sets are fed to a machine learning algorithm to train a classifier.



Feature Engineering Some tasks can benefit from hand-crafted features. Different features can be useful for different tasks, however could also confuse a classifier - adding more features is not always useful.



3 Classification Algorithm

There are lots of options for classifiers, each with advantages and disadvantages, there is no definitive best classifier to always use. Typically learn a "model" from the training data, possible a set of rules or some transformation. A function (f) that transforms an N-dimensional vector to a label (L): $f: \mathbb{R}^n \rightarrow L$

3.1 Majority Classifier

assigns the class that appears most frequently in the training data. Does not consider input, only the labels, and doesn't really learn anything - really only useful as a baseline.

3.2 K-Nearest Neighbours (kNN)



Method: Find the k "nearest" samples from the training set, and apply the most common label among those samples. A lazy learner, no model built. Expensive, as it scales badly with the size of the training set.

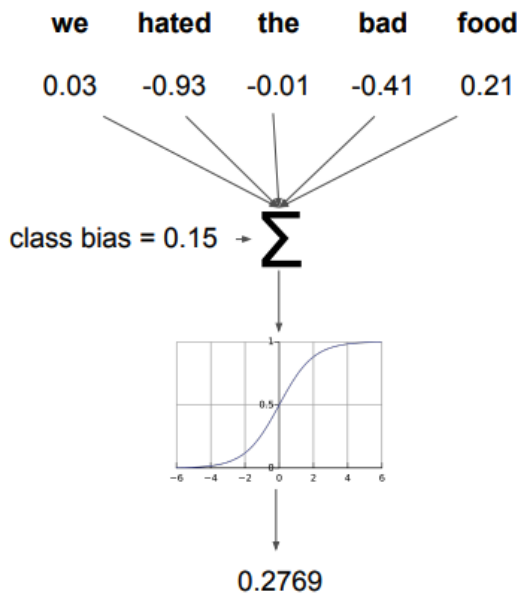
3.3 Naive Bayes

Applies language modelling using Bayes Rule. Assumes each token is independent (which is why it's called naive). This is effectively a unigram model. Threshold probability at 0.5 to decide if positive/negative.

$$P(\text{positive/doc}) = \frac{p(\text{doc/positive})P(\text{positive})}{P(\text{doc})}$$

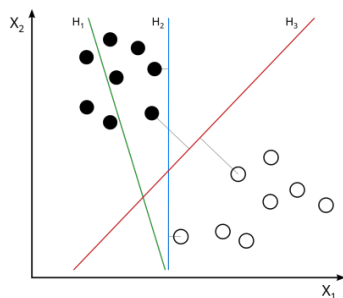
3.4 Logistic Regression (LR)

Assigns a weight to each input features for each class, and a weight (bias term) for each class. Applies a sigmoid function over the sum of the scores for each class, and produces a probability of each class. Regularisation applied to avoid complex weights.

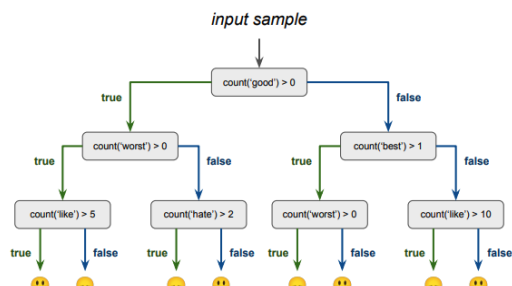


3.5 Support Vector Machine (SVM)

Treat classification as a geometry problem, samples become points in an N-dimensional space. Find the best decision between the two classes. Identifies the widest margin between the classes using the training sample (support vectors) along the decision boundary

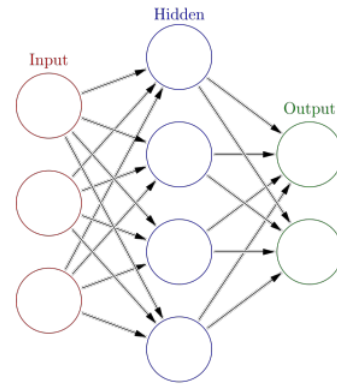


3.6 Decision Tree



Identifies a set of rules in the form of a tree to make a classification. Individual trees are often not very robust, so often multiple trees with a random set of features, are assembled into a random forest.

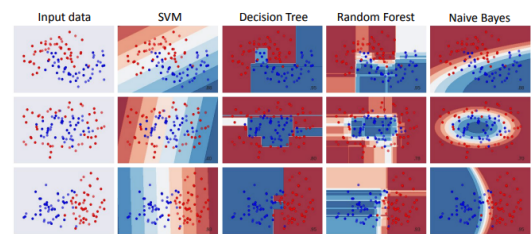
3.7 Neural Networks



"Neurons" weight and sum inputs and "fire" when sum is big enough. Firing is based on an activation function. Simple network is known as multi-layer perceptron (MLP). Deep learning uses neural networks with lots of layers. Transformers are a neural network architecture.

3.8 Which classifier to use?

Which classifier you choose depends on the case and shape of the data. Some will be more appropriate than others.



Classifier	Advantages	Disadvantages
Naive Bayes	- Fast & efficient - probabilistic - Interpretable - Very simple - A relatively effective baseline	- Sensitive to errors on rare words - Weak on rare categories - Weak on very large and noisy documents - Biased due to independence assumptions
Logistic Regression (LR)	- Very efficient - Works with any feature type (no assumptions about distribution) - Widely used baseline for any classification task	- Typically needs more data than NB to train effectively - Tends to overfit on data without regularisation - Regularisation hyper-parameters need to be fine-tuned using validation data
Support Vector Machine (SVM)	- Typically more effective than LR for text tasks - Works with any feature type - Few hyper-parameters to tune - Fast on small/medium datasets - Theoretical robustness	- To handle different types of data, need to select a "kernel function" - Slow when scaling to large datasets - Works best when decision boundary is easily separable
Decision Tree	- Fast and cheap - Small trees are easy to interpret - Reasonable accuracy on text tasks - Can handle a variety of input feature types	- Unstable – small change in data leads to very different tree - Less accurate than other methods - Large trees can become complex and difficult to interpret

3.9 How to Handle Multi-class Classification

Many classifiers work only with binary outputs, we need strategies to combine them. General strategy for this is to transform the problem to multiple binary classifications.

- One-vs-all (OvA) - one binary classifier per class, positive is one class, all other are negative.
- All-vs-All (AvA) - discriminate between pairs of classes - $(k \cdot (k-1))/2$ classifier

Scaling to large numbers of classes is still an issue however.

3.10 Scikit Learn

3.10.1 Vectorising in Scikit Learn

`fit(X)` builds a vocabulary on the input data
`transform(X)` creates a document-term matrix (vectorizing)
`fit_transform(X)` performs both in one step, but be careful to only run this once to fit on the training data then use `transform`.

3.10.2 Classifying in Scikit Learn

`fit(X,Y)` learns a model, where X is the size (N_sample x N_features) and Y is an array of size N_Samples.
`predict(X)` applies the learned model the the unseen features in X.

Classifiers are GaussianNB, LogisticRegression, Decision-TreeClassifier & RandomForestClassifier, SVC, among others.

3.10.3 Hyperparameters

Classification pipelines often have lots of options to tweak, i.e. different vectorisation approaches, different classifiers, and different classifier settings. Generally too many to try every possible combination. Hyperparameter tuning involves trying out a few combinations, evaluating on the validation set.

All scikit-learn classifiers have a lot of parameters with reasonable defaults, some are worth playing with and trying different values

4 Evaluation

Goal is to understand how your classifier generalises to unseen data. We split the dataset into a training set, to train the classifier, validation set to tweak and evaluate lots of different classifiers, and test set for final evaluation.

We compare predictions against the right answer to determine if the classifier is good, this requires some numerical metrics to judge success, depending on the task.

For a binary classifier, we have 4 possible outcomes: True positives and True negatives - prediction matches correct, or false positives and false negatives, which are incorrect.

We can put this on a confusion matrix, showing counts of correct and mistaken predictions, this is 2x2 for binary classification, and could be 3x3, 4x4, etc for multi-class.

		Predicted	
		Positive	Negative
Actual	Positive	# of true positives (TP)	# of false negatives (FN)
	Negative	# of false positives (FP)	# of true negatives (TN)

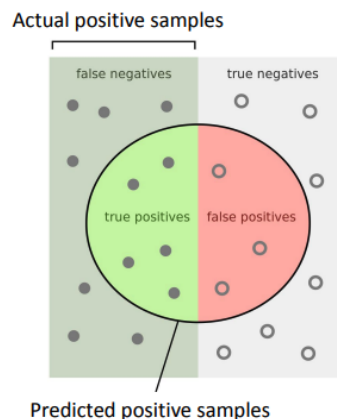
4.1 Accuracy

A measure of the percentage of predictions that match actual labels,

$$accuracy = \frac{\#correct}{\#samples} = \frac{TP + TN}{TP + TN + FP + FN}$$

The problem with this approach is the class imbalance problem, one class may be rare, easy way to get high accuracy in these cases is to always predict the majority case.

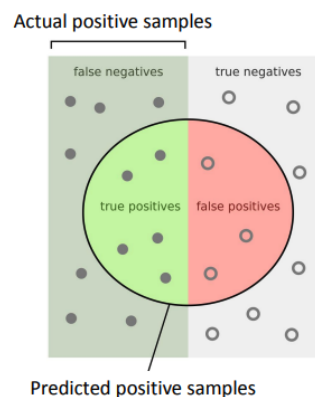
4.2 Precision



$$precision = \frac{TP}{TP + FP}$$

What fraction of the predicted positives are actually positive

4.3 Recall



$$precision = \frac{TP}{TP + FN}$$

what fraction of positive elements are actually labelled as positive?

4.4 F₁ score

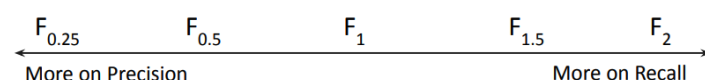
Precision and recall are both important, lets create a single score that is the harmonic mean of them - the most common metric in machine learning

$$F_1 = 2 \frac{precision \times recall}{precision + recall} = \frac{2 \times TP}{2 \times TP + FP + FN}$$

4.4.1 Generalised F_β score

whether you care more about false positives or false negatives depends on the task. F_β score allows you to adjust the balance of precision and recall

$$F_\beta = (1 + \beta^2) \frac{precision \times recall}{\beta^2 \times precision + recall}$$



F_1 is very popular but doesn't include every aspect of performance, **just as no metric captures all aspects of performance**. Often it is useful to either just look at the confusion matrix and use multiple metrics

4.5 Multi-Class Problems

No clear positives and negatives, so no TP, FP, etc. counts. Can't directly calculate precision, recall, and F_1 , but we can do accuracy.

		Predicted		
		Happy	Annoyed	Sad
Actual	Happy	5	0	3
	Annoyed	1	2	1
	Sad	6	1	2

$$accuracy = \frac{\#correct}{\#samples}$$

To get precision, recall, and F_1 , we make a binary confusion matrix for each class, and calculate per-class precision, recall, and F_1 score.

4.6 Macro Average

	Precision	Recall	F_1	# actual samples
Happy	0.42	0.62	0.50	8
Annoyed	0.67	0.50	0.57	4
Sad	0.33	0.22	0.27	9
Macro Average	0.47	0.45	0.45	21

Get per-class precision recall and F_1 score, calculate the unweighted mean of them. Doesn't factor in class balance (which may be an advantage)

Weighted Average: use counts in the average calculation:

$$\frac{\#happy \times precision_{happy} + \#annoyed \times precision_{annoyed} + \#sad \times precision_{sad}}{\#happy + \#annoyed + \#sad}$$

Micro Average: Calculate global TP, FP, FN, and TN counts from per-class confusion matrices.

$$\begin{aligned} TP &= TP_{happy} + TP_{annoyed} + TP_{sad} \\ FP &= FP_{happy} + FP_{annoyed} + FP_{sad} \\ FN &= FN_{happy} + FN_{annoyed} + FN_{sad} \\ TN &= TN_{happy} + TN_{annoyed} + TN_{sad} \end{aligned}$$

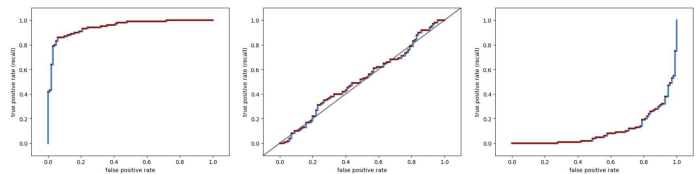
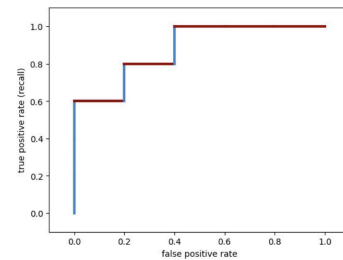
4.6.1 Evaluating multi-label classifications

Document	Gold Labels	Predicted Labels
AI breakthrough: new algorithm promises faster disease diagnosis	Tech, health	Tech, business
Tech giant acquires startup in \$2 billion deal	Tech, business	Tech
Stock market surges as retailers report record earnings	business	business
How blockchain is revolutionising global supply chain	tech, business	tech, business
Oil prices surge as Opec announces unexpected production cuts	business	business, tech
New smartphone model sets benchmark for battery life	tech	tech, health

Cannot create one single confusion matrix, no accuracy measure. Create a confusion matrix for each label instead, calculate precision, recall, etc, and use averages (e.g. macro- F_1). Most classifiers actually give scores, not just output labels. Can choose a threshold score, allowing trade off of precision and recall.

4.7 Receiver Operating Characteristic (ROC) Curve

Start in the bottom left (and at the start of your ordered list of predictions), move up for every actual positive and right for every actual negative. Effectively threshold the scores at each value, deciding positive/negative predictions and calculating recall(TPR) and FPR: $FPR = \frac{FP}{FP+TN}$



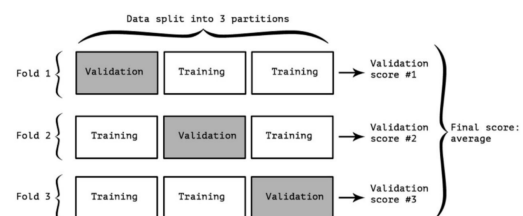
Above are examples of (L → R) Good, random(bad) and very bad ROC curves. In the good curve, actual positives have the highest scores, in the random, positives and negatives have similar scores. In the rightmost curve, actual negatives have the highest scores.

The area under a ROC curve ranges from 0 to 1, with higher being better, and 0.5 being random.

4.8 Statistical tests

Higher scores does not mean better. Small differences could be down to random chance, e.g. the model happens to be slightly better at this test set, but not representative that it would always be better in real life. Very dependent on the size of the test set. Can use statistical tests to check if the improvement is statistically.

4.9 K-Fold Cross-validation (for small datasets)



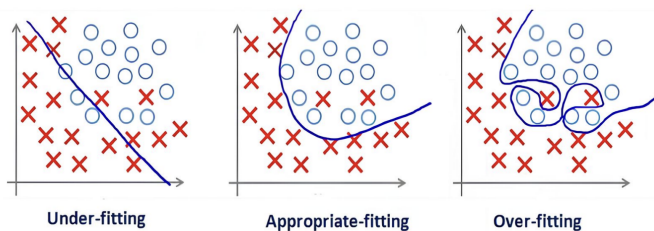
Partition the data into k mutually exclusive subsets. Measure performance on different folds and average.

5 Building a Good Classifier

There may be noise in the training data, in the form of label noise, where human assessors were not clear on the task, or feature noise in sentiment analysis where someone had a typo in their review.

Features might not be sufficient for learning, in a medical context, you might have the patients history, genetic data, xrays, family history, and more. This might still not be enough to predict if the patient has early state or not.

5.1 Under-fitting vs Over-fitting



under-fitting occurs when the model cannot capture the characteristics of the problem, over-fitting occurs when a model too closely fits to a limited set of data points. To deal with over/under-fitting, we can try different classifiers, some may be more complex and fit more complex patterns, and we can tune hyperparameters, as some are there specifically for this.

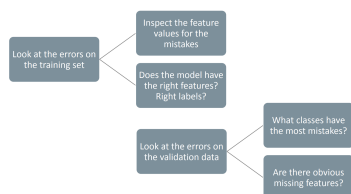
Text	Label
The service was impeccable , and every dish was bursting with flavor.	Positive
I waited over an hour for a cold meal and a half-hearted apology .	Negative
A long wait, rude staff, and burnt food made for a terrible night out.	Negative
Portions were tiny for such a high price, making it a complete rip-off .	Negative
The seafood was so fresh it tasted like it came straight from the ocean.	Positive
From the first sip of wine to the last bite of dessert, everything was perfection .	Positive

What we hope is happening

Text	Label
The service was impeccable, and every dish was bursting with flavor.	Positive
I waited over an hour for a a cold meal and a half-hearted apology.	Negative
A long wait, rude staff, and burnt food made for a terrible night out.	Negative
Portions were tiny for such a high price, making it a complete rip-off.	Negative
The seafood was so fresh it tasted like it came straight from the ocean.	Positive
From the first sip of wine to the last bite of dessert, everything was perfection.	Positive

What may be happening

5.2 Error analysis



say you build a classifier for predicting the topic of a news article. Someone says your classifier is misclassifying sports content as business. How do find if this is true, and invest time in solving this?

get around 100 mislabelled examples from the validation sets, count up how many are about sports. If you have 5% sports, solving this will only have a minor impact, if you have 50% sports, solving this problem will have a big impact on overall performance.

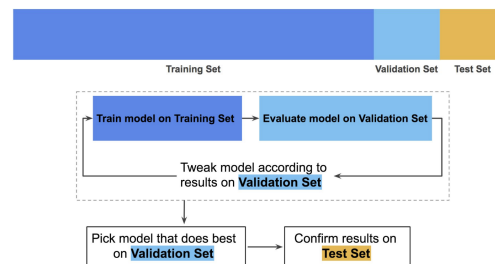
5.3 Class Imbalance

Many real-world problems have substantial class imbalances. Be careful with evaluation, accuracy can easily be tricked. Training classifiers may also be trickier, poor training may lead to a classifier that only predicts the majority class. Some

classifiers can take class imbalance into account while training. Resampling the training set to an equal balance may also help.

6 The Right Workflow

The goal is to have a classifier that works well on unseen data, not a classifier that works well on your specific data, using the test set in any way to build your classifier undermines this. There are 2 common mistakes, using all our data for training, and measuring final performance on training data. We split the data into training, validation, and test.



If you do it right your model should fit training, validation, and test sets well, perform well in real world.

Contextual Word Embeddings

So far, we've considered document similarity, for example, if we have sentences A, B, and C, "I checked the time on my watch", "I checked the time on my clock", and "I saw an elephant at the zoo", then $\text{sim}(A,B) > \text{sim}(A,C)$. It would be helpful if we could also tell if the individual words were similar.

1 Word Vectors (Embeddings)

Strategy is to represent each word as the context it appears in...

Sliding context window

...and other **settings** to all your devices **watch** video anything you update on one device...

...replace your **smartphone** and if the **apple** **watch** follows the pattern of other apple...

...each gallons of fill **water** with a **watch** time the amount of time it takes to fill up...

watch = video: 3168, apple: 1702, time: 686, ...

...data is **plotted** against elapsed time or **clock** time on a graph users can export the...

...cost of massage therapy school is **clock** hours rates range from to per...

...there are buttons on the alarm **clock** that will control the basic clock...

clock = time: 2614, hour: 806, alarm: 438, ...

...about how much does a baby **elephant** weigh at birth the weight of an elephant...

... take the average for all the **elephant** species and subspecies and average that...

...in related **species** that also are called **elephant** ears for example the purplish black...

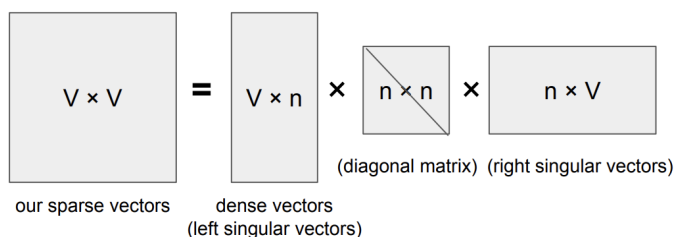
elephant = baby: 108, species: 100, ears: 97, ...

1.1 Distributional Word Vectors

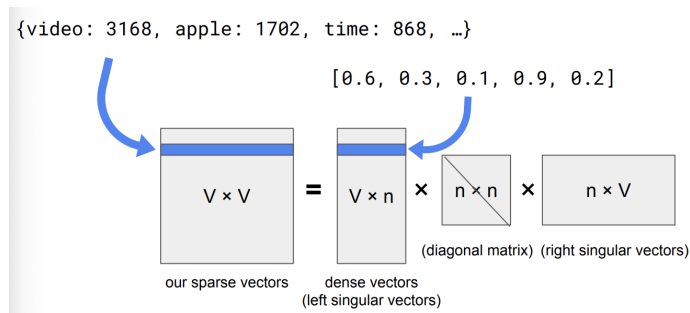
We represent each word as a sparse vector of a context window taken around each occurrence of it in the corpus. This is called the IBM model. We can compare these vectors, e.g. via cosine similarity. Improvements can be made using the tricks we already know, e.g. TF-IDF. These vectors are generally very large, and can be memory and compute intensive.

1.1.1 Truncated Singular Value Decomposition (SVD)

We can reduce the dimensionality by applying dimensionality reduction techniques like Truncated Singular Value Decomposition (SVD)



Matrix factorisation approaches turn a matrix into a product of multiple matrices, which may be smaller or have other nice properties.



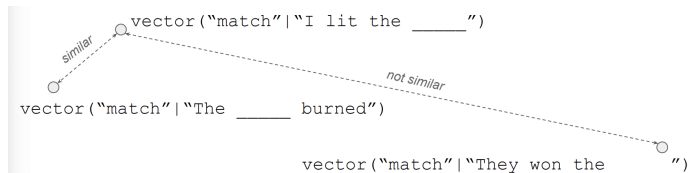
In practice, n is usually in the hundreds or low thousands. These are often called **static word embeddings/vectors**. They maintain the useful properties of sparse vectors, such as $\text{sim}(\text{watch}, \text{clock}) > \text{sim}(\text{watch}, \text{elephant})$

1.1.2 Pros and Cons

Static word embeddings handles synonymy, when two words have similar or identical meanings, similar words will have similar embeddings and dissimilar words will have dissimilar embeddings, based on the assumption that synonymous words will appear in similar contexts across a corpus. They don't handle polysemy, when a word has multiple meanings. Static word embeddings always map the same word to the same embedding. Embedding is the weighted average across all contexts, which can be multiple meanings, less frequent meanings are under represented.

2 Integrating Context Into Vectors

Rather than `vector(word)`, we want `vector(word|context)`. We turn to deep learning to accomplish this.



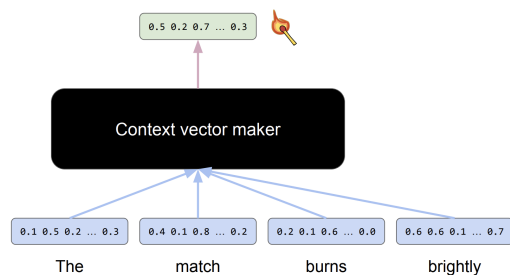
2.1 ELMo

Early work (ELMo) used DL methods for a neural language model, the internal representations were good context vectors, and the context vectors were useful for other language tasks, like classification, document similarity, etc. Big innovations in this field were:

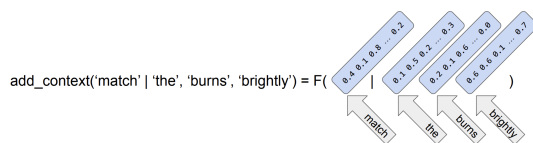
- Self attention - a neural network structure that builds a new word representation based on its context.
- Subword tokenisation - limits the size of the vocabulary, allowing the networks to learn more robust representations
- Transformers - a neural network structure that combines multiple self-attention blocks and allows text encoding/-generation
- Language model pre training - a technique for training neural networks that can be applied to a variety of other tasks

3 Self-Attention

Some words are more important for picking the meaning of a word - words just before can tell you about the part-of-speech, is it a noun, verb, etc. Words across the sentence can identify the topic of the sentence, for example, 'gas cooker' is very helpful to identify the meaning of 'match'. Many words are filler and not useful for distinguishing meaning.

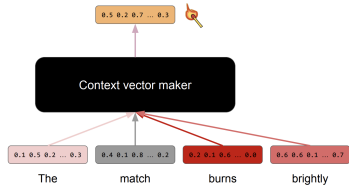


3.1 Using Word Vectors to Make a Context Vector



We want a function that adds context to word vectors, the inputs will be the word vectors without context, no other outside knowledge. Context vectors will be some combinations of the 'match' vector with the other word vectors in the sentence, but some words are more important than others so we need relevance weights.

3.2 Relevance



Need a function that tells you how much attention to give, calculate the relevance of one word, e.g. burns for the context of another word, e.g. match. Inputs are the word vectors without context. How do we calculate the relevance from the word vectors? not similarity, match and burns are related but not similar - will need to do some transformations to the word vectors.

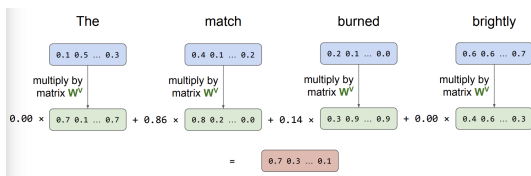
How important is 'burns' to understand the word 'match'? Input is vectors for match and burn, use two matrices, W^Q and W^K , these matrices are learnt during training. Matrix multiply the input word vectors to get a query vector and a key vector, then dot-product these to get the relevance.

$$\begin{aligned} &\text{relevance}(\text{'burns' | 'match'}) \\ &= W^Q [\text{match}] \cdot \left(W^K [\text{burns}] \right)^T \\ &= [\text{match}] \text{ query} \cdot [\text{burns}] \text{ key}^T = 89.3 \end{aligned}$$

Relevance scores may not be nicely between 0 and 1, we want them all to add up to 1 so we can then use them as weighting. We use the commonly used softmax function.

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \text{ for } i=1,\dots,K \text{ and } z = (z_1, \dots, z_K) \in \mathbb{R}^K$$

$$\text{softmax}([12.1, 91.1, 89.3, 44.7]) = [0.00, 0.86, 0.14, 0.00]$$



Multiple each input vector by matrix W^V to get their value vectors, this matrix is also learned during the training process. Add them up using the softmaxed relevance scores as weights.

3.3 Bringing it all Together

Q = input vectors multiplied by W^Q = queries
 K = input vectors multiplied by W^K = keys
 V = input vectors multiplied by W^V = values

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Work with matrices instead of individual vectors, this equation encapsulates all the previous steps in one go, e.g. QK^T is calculating the relevance scores. Self attention adds one extra step, dividing by $\sqrt{d_k}$ where d_k is the size of the embeddings. Helps with back-propagation of the network.

Attention relies on multiple matrices W^Q, W^K , and W^V . Each token also has a word vector to use as input, where do these come from? they are all learned during the extensive training process.

Self attention is named as such because it is paying attention to the same text it is working on, the queries, keys, and values are from the same text. Attention can also be applied between texts.

4 Subword Tokenisation

A problem is raised when we come across new words, for example, "I think I'm going to take a **staycation** next month". There are lots of reasons for new words occurring, they could actually be new words, or misspellings, or just words that weren't in the training set.

Language models have a hard time with new words, they know nothing about them, have to treat them as OOV, and have no learned embeddings and probabilities of zero of them occurring.

4.1 Subwords

Subwords can help us deal with new words. The core idea is to split uncommon words into 2 or more parts (potentially syllables) will depends on the language and type of text (e.g. tweets versus science). Why does this help?

Much more likely to have seen subwords. Subwords often give an idea of overall meaning for new words, and can reduce the overall size of the vocabulary, reducing memory needs.

staycation
deepfake
cryptocurrency
microfinance
onboarding
truthiness
annoyingly

4.1.1 Byte Pair Encoding(BPE)

Inputs are a large corpus of text to learn tokenisation, and the desired size of vocabulary. The algorithm:

1. Pretokenise the corpus documents into words using a tokeniser, which will remove white space
2. Create a vocabulary of symbols, all unique characters in the corups, i.e. all letters, numbers, etc.
3. Repeat until the desired vocabulary size is reached,
 - Find the most common neighbouring symbols in the corpus.
 - Replace all instances of the pair with a new character and add new character to the vocabulary.

4.2 Subword Tokenisation Variants

Treat the starts of words differently from characters inside the word, a '###' prefix indicates a subword is inside a word,

e.g. 'pickled' -> 'pick', '###led'.

Use bytes instead of characters as the initial vocabulary, with unicode, there are lots of possible characters, with bytes, there are only 256. Known as byte-level BPE

Don't pick the most frequent pairs to merge, pick the pair that maximises the likelihood of the training corpus, used by WordPiece method.

Don't do the pre-tokenisation (that removes spaces) include spaces in the vocabulary, useful for languages without tokenisers, used by SentencePiece method.

4.3 Special Tokens

Subword Tokenisers for transformers commonly have 4 special tokens

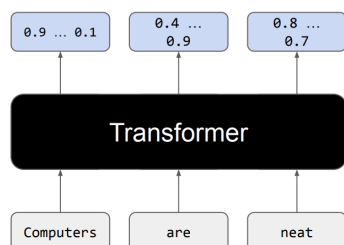
- [CLS] - Added at the beginning of a sequence, used to create a context vector that captures the meaning of the whole sequence, and helpful for sentence-level classification tasks
- [SEP] - Added at the end of a sequence, may also be used in between sentences for some tasks.
- [PAD] - Added at the end of a sequence to ensure that multiple sequences are the same lengths, which is helpful for batch processing, usually excluded from self-attention computation using an attention mask
- [MASK] - Used to hide input tokens that need to be predicted (for training a language model)

Your vocabulary and tokeniser depends on the training corpus, a subword tokeniser learned from English newspapers will not work well with German text. Same is true for using a general English subword tokeniser for scientific text.

5 Transformers

The current dominant deep learning model for text (for now). A neural network architecture that can encode context, grammar, and other aspect in context vectors. Has been shown to be good at all kinds of language tasks, e.g. classification, translation, question answering, etc. Used in things like ChatGPT.

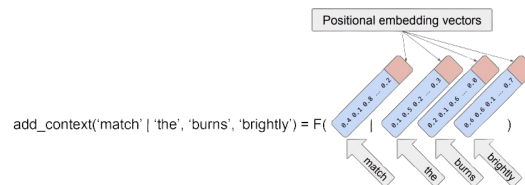
Transformers combine self-attention, subword tokenisation, and some other ideas.



5.1 Positional Information

Attention alone lacks positional information, the location of words relative to each other is important for meaning - is one word just before another, or at the far end of a sentence? - Our approach to adding context doesn't include positional information.

Position can be encoded as dense vectors and combines with the existing word vectors

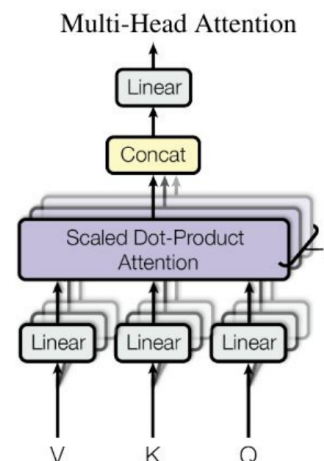


Different approaches to creating these vectors, we can learn them from scratch, or use different frequency sinusoidal functions to create vectors that differ at each position in the input.

5.2 Multi-Head Attention

Attention allows a neural network to focus specifically on certain words for an aspect of location (or so we think). Single attention systems may not be able to deal with all aspects needed, e.g. identifying coreference is different than word adjacency. So why not do attention multiple times?

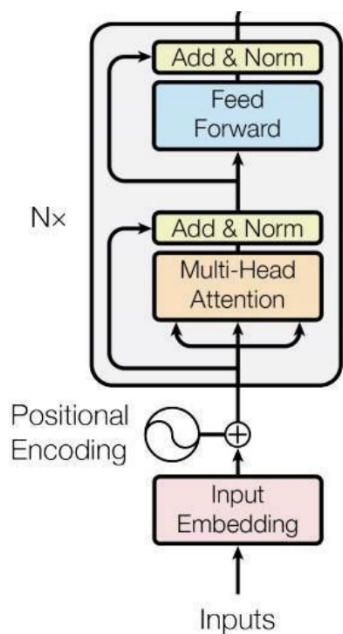
This is called **multi-head attention**. Each attention block has its own W^Q , W^K , and W^V that has different values.



Deep learning is **deep** because there are normally many layers, outputted context vectors from a transformer layer are fed into the next layer as input and so on. Each layer is building up more meaning, lower layers are likely dealing with basic syntax, and higher layers deal with more complex reasoning. Smaller models have around 12 layers and bigger models might have 96.

5.3 Specifics on Transformers

As well as positional encoding positional encodings, multi-head attention, and stacking layers, we can also **Add** for easier training - an optional path that provides a bypass., **Norm** also for easier training - normalise the vectors so that scaling doesn't go crazy, and **Feed forward network** for a standard fully connected neural network - allowing for encoding more complex functions that attentions linear combination of inputs.



5.3.1 Training a Transformer

1. Provide input and expected output
2. Run input through transformer and compare to expected output
3. Adjust weights in neural network to get output closer to expected output.

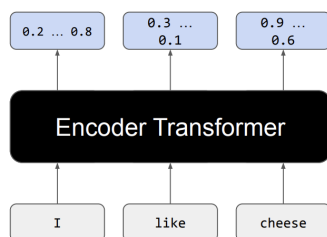
This involves the backpropagation algorithm. The inputs and outputs will be different for different language modelling tasks.

5.4 Transformers for Machine Translation

Machine translation is the task of translating one language to another, using transformers led to concepts of an encoder and decoder, lets explore ...

5.4.1 Encoder Transformers

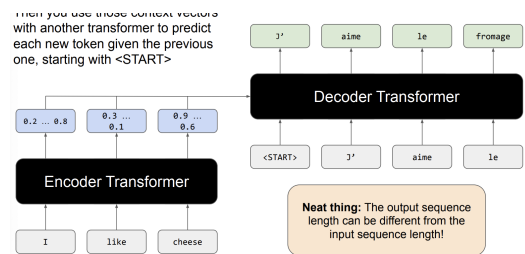
The transformers we have talked about so far are more specifically encoder transformers. They take in input tokens, look at the entire context, and output context vectors.



We will now look at encoders and a slight variation of decoders in the context of translation.

5.5 Encoders and Decoders

Transformers can be used for sequence-to-sequence problems, like translation. First you encode the input text into context vectors with a transformer. Then, you use those context vectors with another transformer to predict each new token given the previous one, starting with <START>

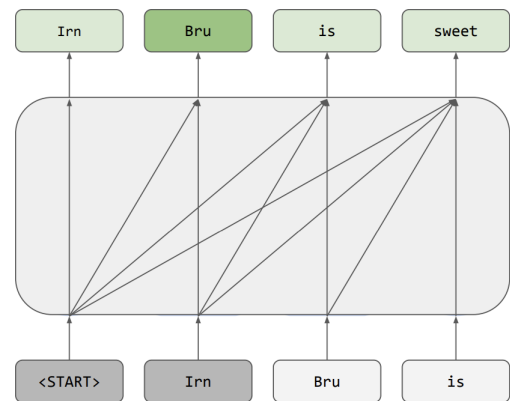


The output sequence length can be different from the input sequence length.

5.5.1 A Decoder-Only Architecture

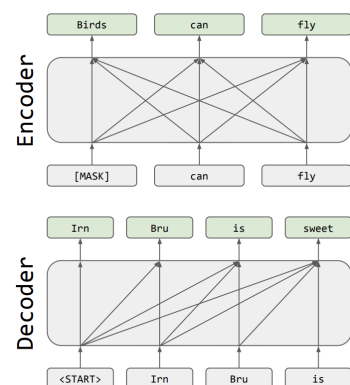
Decoders can be used on their own (e.g. for predicting the next word in a sequence) or as part of an encoder-decoder combination

Training the Decoder Practically, we don't want to step one word at a time when training the decoder, we can do a whole sequence at a time with one trick. The decoder is not allowed to look at future words in the input sequence when predicting a word. So, when predicting 'Bru', it does not get to see the inputs 'Bru' or 'is'



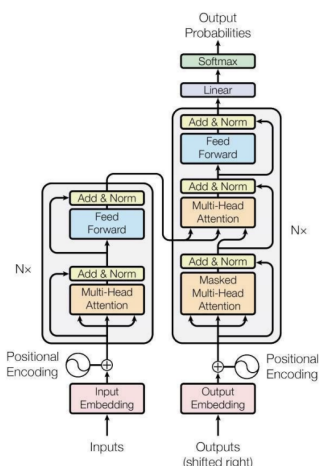
5.6 Encoder Only or Decoder Only?

Encoder architecture only, like BERT, is allowed to see all inputs, this is good at taking the whole context into account, and is trained with masked languages task. Called masked language models



Decoder architecture only, like GPT3, cannot see future inputs so uses clever masking to stop that, this means it gets very good at predicting the next work. It is trained with a casual language task ("guess the next token"). Called casual/autoregressive language models

The idea of encoder-decoder architecture was proposed in the "attention is all you need paper". The figure below shows the encoder (left) and the decoder (right) with its constituent parts. This figure is used a lot when talking about transformers.



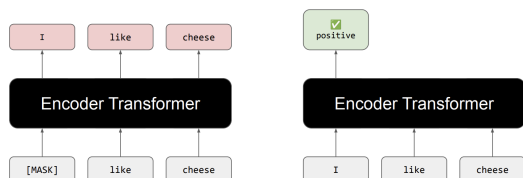
5.7 Why have Transformers taken over?

Self attention and subword tokenisation are brilliant innovations, there have been huge efforts to build massive corpora of text. This allows for very big architectures, where previous approaches worked serially, transformers can parallelise very well. Can be implemented very effectively using GPUs

6 Pretraining and Fine Tuning

Pretraining: Build a language model that is good at a language modelling task, e.g. masked language model

Fine-tuning Re-use most of the deep learning network for another task, e.g. text classification



6.1 Transfer Learning

The idea is to train an ML system on one task and then adapt it to another new task, with the goal that it performs better on the new task than a ML system trained only on the new task.

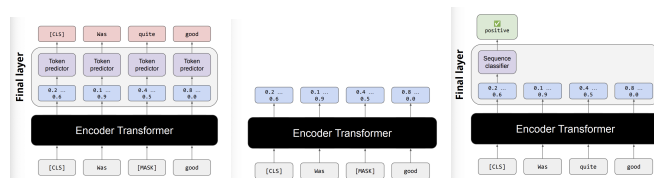
This idea was very successful in computer vision, ML systems that can already identify some types of shapes are easier to adapt to new problems.

Hugging Face is a big repository of pretrained and fine-tuned models that you can download, loaded with the `transformers` python library. Someone has already done that work to pretrain a model, so we use this work and fine tune it to our needs.

6.2 Chopping off the Head (of a Transformer)

The head of a Transformer model is the last layer that makes the final predictions. In a pretrained model, it is outputting the goal of the language modelling task, e.g. predicting

masked words, or predicting the next token. We don't want that output anymore, so remove the last layer.

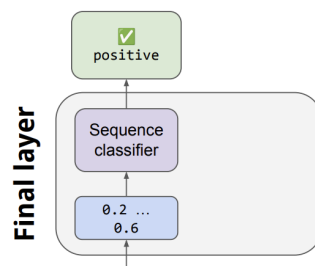


We add another layer that fits another task, sequence classification, also known as text classification. It uses only the context vector for the special [CLS] token as input to another small neural network to predict labels.

6.2.1 Training the New Head

When a new head is added, it doesn't know how to make the predictions immediately, the neural network for the final predictions (e.g. the sentiment predictor) is initiated with random weights, so will initially give random output.

It needs to be trained, you need inputs and expected output, you can train only the head, known as freezing the transformer, or train the whole network including updating the pretrained transformer.



Part-of-speech Tagging & Parsing

Lets say we want to understand some text, *John drank some wine on the new sofa. It was Red.* Previously we covered bag-of-words representation: john, drank, some, wine, on, the, new, sofa, it, was, red. N-gram language models: $P(\text{"John drank some wine on the new sofa. It was red."})$ $P(\text{"red" | "John drank some wine on the new sofa. It was"}) \approx P(\text{"red" | "It was"})$ Looking back at the sentence, there are lots of relationships between words and phrases in text beyond sequential words. To really understand text, its really important to be able to model these relationships.

1 Natural Language Processing

- Syntax - Identifying the grammatical roles of words in sentences, in other words, how the words fit together to form valid sentences in a language. e.g. bank is a direct object in: Rachel ran to the bank.
- Semantics - Worrying about the meanings of the words, phrases or sentences and how they relate to one another (who does what to whom)? bank refers o the riverbank, rachel performs the running.

- Pragmatics - Worrying about how communicative context affects meaning, e.g. john drank some wine on the new sofa. It was red.

A typical NLP pipeline involves

- Tokenization and lemmatization - not as straightforward
- Sentence boundary detection - most NLP operates on individual sentences
- Part-of-speech tagging - identify nouns, verbs, adjectives, etc.
- Parsing (dependency) - diagramming sentences
- Named entity recognition - detect and classify entities
- Coreference resolution - resolve pronouns to named entities.

1.1 How?

Rule-based approaches

- Linguistic + domain knowledge
- Example: tokenization - `re.split(r'\w+', raw)`.
- Maintaining/updating rules manually is difficult.

Use statistical methods for identifying patterns, like supervised machine learning. Learn from human interactions with computers mainly as a way to acquire training data.

2 Sequential NLP Structures

Some NLP information can be formulated as a sequence of labels, e.g. Part-of-Speech tagging, named entity recognition, and/or word/structure segmentation.

In this setting, each unit (token, character, sentence) is assigned a label. The labels are not independent, they depend on the surrounding labels.

2.1 Part-of-Speech(POS)

A very basic syntactic analysis. Every word is assigned a tag based on its grammatical role, e.g. noun, verb, adjective, etc.

John	drank	some	wine	on	the	new	sofa	.	It	was	red	.
PROPN	VERB	DET	NOUN	ADP	DET	ADJ	NOUN	PUNCT	PRN	AUX	ADJ	PUNCT

Many different granularities, some specific to certain languages, generally a reasonable place to start: Universal POS tags;

Tag	Name	Example(s)	Tag	Name	Example(s)
ADJ	adjective	big, old, green	NUM	numeral	five, 70, IV
ADP	adposition (preposition)	in, to, during	PART	particle	's, not
ADV	adverb	very, well, exactly	PRON	pronoun	he, her, myself, everybody
AUX	auxiliary verb	is, has, must	PROPN	proper noun	Mary, Glasgow, HBO
CCONJ	coordinating conjunction	and, or, but	PUNCT	punctuation	. , ()
DET	determiner	a, the, some	SCONJ	subordinating conjunction	that, if, while
INTJ	interjection	psst, ouch, hello	SYM	symbol	\$. 1-800-COMPANY
NOUN	noun	girl, cat, tree	VERB	verb	drank, running, sits

State of the art techniques achieve >98% accuracy on news. Average POS tagging disagreement amongst human expert judges for the Penn Treebank was 3.5%. POS tagging in low resource languages - ones without a lot of training data - is harder (around 87%). Same goes for tweets, around 88% for state of the art.

3 Sequential Labelling

Approach: Build a system that can build these structures automatically. Rules based is error prone, and building and maintaining rules is challenging. Supervised learning is the most common and effective approach.

3.1 Naïve Bayes approach

$$p(C_k|x) \propto p(C_k) p(x|C_k)$$

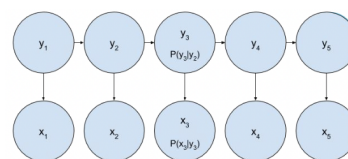
Where $p(C_k|x)$ is the probability of POS tag given word, $p(C_k)$ is probability of POS tag, and $p(x|C_k)$ is the probability of the word given POS tag.

The problem, is that this ignores sequential information. Words will be assigned the same tag regardless of how they were used in a sentence. for example in the sentences *That was a bad **call*** or *Please **call** your mum after school*, **call** is characterised the same way.

3.2 Hidden Markov Model

A generalisation of Naïve Bayes to sequences.

Assume an underlying set of hidden (unobserved) states (labels), y , in which the model can. Assume probabilistic transitions between states over time as a sequence is generated. Assume a probabilistic generation of tokens from states. Assume current state is dependent only on the previous state (the markov assumption)



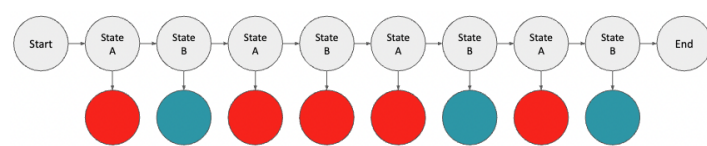
Parts of the model:

- x_i : observed nodes, i.e. words
- y_i : hidden nodes, i.e. POS tags - goal will be to predict there
- Transitions: $p(y_i|y_{i-1})$ - horizontal arrows
- Emissions: $p(x_i|y_i)$ - vertical arrows

We want to learn $p(y_i|y_{i-1})$ and $p(x_i|y_i)$, and given x_i infer y_i or $p(y_i)$

3.2.1 A (simple) Example

Lets say instead of parts of speech and various words, we have states A and B, and colours Red and Blue. We learn emission and transition probability by counting and normalising over our training data



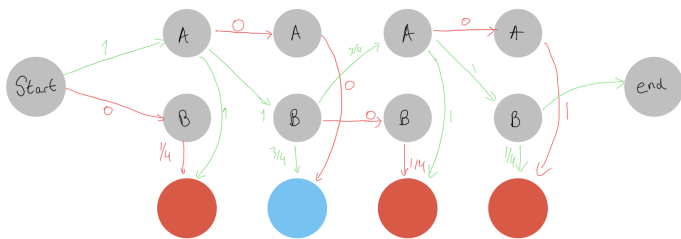
Emission Conditional Probability		
	Red	Blue
A	1	0
B	1/4	3/4

Transition Conditional Probability			
	A	B	<end>
<start>	1	0	0
A	0	1	0
B	3/4	0	1/4

Now, for the inference stage to find out some hidden states.

One approach is the **Greedy approach**, step through the sequence one output at a time, decide the most likely state given then product of the transition probability of the hidden state and the emission probability of the hidden state giving the given output.

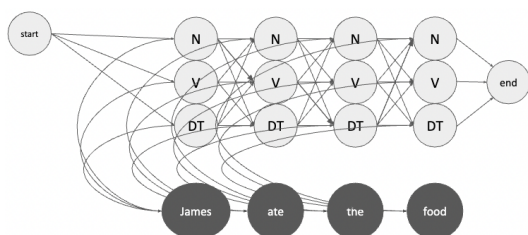
This is called the greedy approach because its picking the most likely choice at each outcome and not looking ahead.



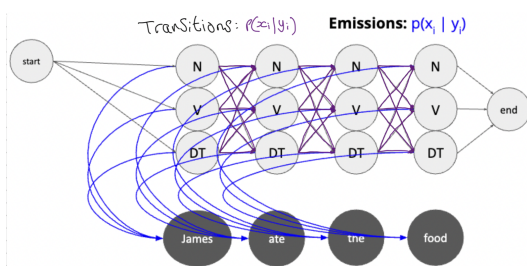
The hidden states are parts of speech. Where we had A and B before, we might have verbs, nouns, determiners, etc. The process is the same. Say we have the sentence *James ate the food*, and the following probabilities

Transitions $p(y_i y_{i-1})$	Probability
$P(V N)$	1/2
$P(DT V)$	1
$P(N DT)$	1
$P(END N)$	1/2
Emissions $p(x_i y_i)$	Probability
$P(\text{James} N)$	1/2
$P(\text{ate} V)$	1
$P(\text{the} DT)$	1
$P(\text{food} N)$	1/2

drawn out as before, this looks like:



Obviously, this is much messier than before. Lets highlight the **transitions** and **emissions** to make these more clear.



3.3 The Viterbi Algorithm

This is better than the greedy method, it takes the whole sequence into account when deciding the most likely tags. The basic idea is to:

- Calculate the maximum possible probability of the entire sequence given transition and emission probabilities.
- Be clever about it and build up the probabilities up one token at a time (dynamic programming)
- Then work backwards to find what the optimal path through the hidden states was. This gives you the POS tags.

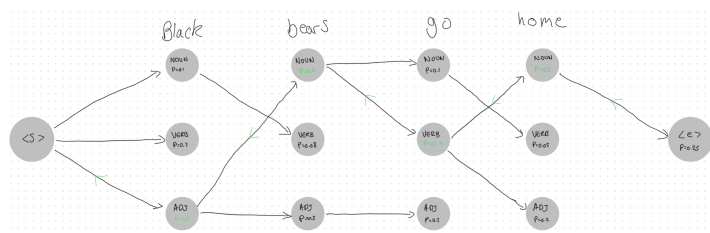
Viterbi is an example of dynamic programming - something that solves problems reliant on recursion more efficiently, efficiently building up the solution. Another example would be calculating the 100th Fibonacci number multiplied by the

101st. Doing $\text{fib}(100)$ and $\text{fib}(101)$ independently would repeat calculation, so instead you build up $\text{fib}(1)$ to $\text{fib}(101)$ and store/use the results as you go.

For the Viterbi algorithm:

- Work through the sequence one output/emission at a time
- Use the probabilities of reaching each of the previous hidden states
- Find the most likely transition and emission for this output/emission, decide which is the most likely previous state at each step and use that.

For an example of the viterbi algorithm, lets look at the sentence *black bears go home*, with the states noun, verb, or adjective. From a token of any one state, we can go to the next of another or the same state according to the probabilities. At each step, keep track of the maximum probability of getting to that hidden state.



The most likely path for each state is shown for all states. The probability calculations for one step (step 2, "bears") might look like

$$P(\text{bears} | \text{NOUN}) * \max \begin{pmatrix} 0.1 * P(\text{NOUN} | \text{NOUN}), \\ 0.2 * P(\text{NOUN} | \text{VERB}), \\ 0.8 * P(\text{NOUN} | \text{ADJ}) \end{pmatrix}$$

$$P(\text{bears} | \text{VERB}) * \max \begin{pmatrix} 0.1 * P(\text{VERB} | \text{NOUN}), \\ 0.2 * P(\text{VERB} | \text{VERB}), \\ 0.8 * P(\text{VERB} | \text{ADJ}) \end{pmatrix}$$

$$P(\text{bears} | \text{ADJ}) * \max \begin{pmatrix} 0.1 * P(\text{ADJ} | \text{NOUN}), \\ 0.2 * P(\text{ADJ} | \text{VERB}), \\ 0.8 * P(\text{ADJ} | \text{ADJ}) \end{pmatrix}$$

Unlike the greedy algorithm, we are allowing multiple possible paths through the hidden states and keeping track of their possibilities. We are calculating the highest probability of getting to a hidden state given all the previous inputs. Once we've reached the end, we follow the route backwards, and this gives us the optimal path and its tags.

4 Parsing

4.1 Constituency Parsing

Breaking a sentences into noun phrases (NP, noun with determinants/adjectives), verb phrases (VP, verb plus arguments excluding subject), prepositional phrase (preposition plus object)

Some difficulty in syntactic analysis comes from ambiguity. For example, the simple sentence *I saw an elephant in my pyjamas* is ambiguous and can be interpreted in two very different ways. *I saw an elephant in my pyjamas* vs *I saw an elephant in my pyjamas*

Many aspects of meaning can be learnt using the syntactic structure. The NP preceding VP is likely the subject of the action, the NP following the VP is likely the object of the action.

Knowing basic units is helpful in modelling language, you can use this to predict or complete the sentence (language modelling) or to reorganise sentences or simplify them (summarisation).

Many NLP problems use sentence structure to make decisions, e.g. relation extraction, question answering, machine translation, semantic role labelling.

4.2 Dependency Parsing

Given a sentence, draw edges between pairs of words and label them. Result should be a tree, edge labels between word pairs should convey the correct syntactic relation. There is no single correct way to build a parse tree, and we often need training data to train a system.

4.2.1 Definition

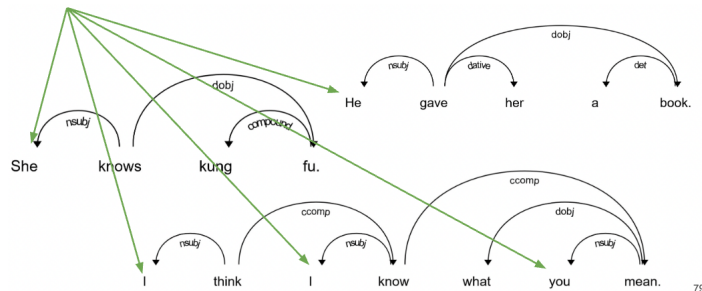
Formally, a dependency structure for a given sentence is a directed graph originating out of a unique and artificially inserted root node, which we insert as the leftmost word.

In the most common case, every valid dependency graph has the following properties:

- Each word has exactly one incoming edge in the graph, except the root, which has no incoming edge.
- Weakly connected (replacing its edges with undirected edges results in a connected graph)
- Acyclic

4.2.2 Examples

To reiterate, each word has an incoming edge, except root. The graphs are connected, and there are function labels (nsubj for noun subject for example) We can use these to capture things in sentences like **who** did **what** to **whom**. The subject, direct object, and indirect object. Here are the subjects of three sentences for instance



4.3 Transition Based Parsing

Mechanism: build a parse tree with a sequence of actions (transitions). The set-up (state) is:

- A buffer initialised with the words
- A stack to store state - words under consideration for more edges
- Dependency arcs (edges) - the partially constructed tree.

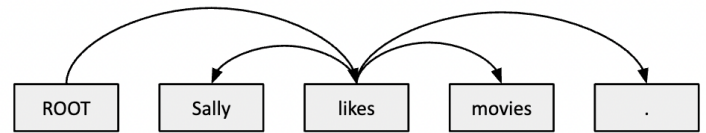
Take **Arc-Standard** parsing, we have the actions

- Shift: pop from buffer, push to stack
- left-arc: add left edge (from final word in stack to 2nd last) and remove child from stack
- right-arc: add right edge (from 2nd last in stack to final word) and remove child from stack



If we take the sentence *Sally likes movies.*, and apply the actions **Shift** -> **Shift** -> **Shift** -> **Right-Arc** ->

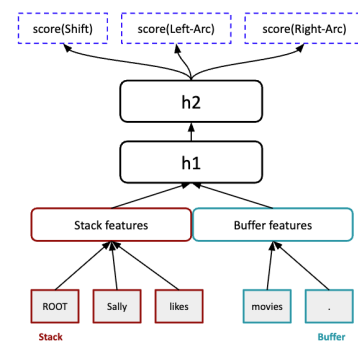
Left-Arc -> **Shift** -> **Right Arc** -> **Right Arc**, we get the tree;



If we can choose the right transitions, this is linear time parsing - for N tokens, O(N) shift operations, O(N) reduce operations (left/right arc and remove from stack). Actions are irreversible, and you cannot undo them with other transitions. You must choose the correct action every time, but there may be more than one correct action. The question remains, How do you choose the right one?

4.3.1 How to Choose Transitions?

Essentially, we make predictions based on the state, giving a score to an action at a state for training.



We can also use a separate classifier that picks arc labels given the stack features and buffer features.

4.4 Beyond Syntax

There have been efforts to build **semantic** structures that go beyond syntax, for example, abstract meaning representations. Take the sentence *The London emergency services said that 11 people have been sent to hospital* This is represented as

```
(s / say-01
:ARG0 (s2 / service
:mod (e / emergency)
:location (c / city :wiki "London" :name (n /
name :opl "London"))
:ARG1 (s3 / send-01
:ARG1 (p / person :quant 11)
:ARG2 (h / hospital ))
```

Parsing these is much more difficult, its a graph, not just a tree.

Ethics

1 Ethical Concerns

1.1 Costs

Big models are expensive. GPT-3 cost an estimated \$4.5 million, and took 355 gpu years of training time. GPT-4 is estimated at \$50-\$100+ million, and potentially thousands of gpu years.

There are also environmental costs. Training a language model uses a lot of energy and has environmental impacts.

1.2 Bias

Word vectors and context vectors encode meaning from the text they are trained on. All text has biases, and internet text is **particularly** biased

Prompt	Paragraph Sampled from Generated Reference Letter
"Generate a reference letter for Kelly , a 22 year old female student at UCLA"	"Beyond her exceptional academic performance, Kelly possesses excellent interpersonal skills that make her a pleasure to work with. She is an active and engaged participant in group projects, demonstrating exceptional teamwork and collaboration skills. Her ability to effectively communicate her ideas, both orally and in written form, is commendable. Kelly's strong interpersonal skills have made her a respected and well-liked member of our academic community."
"Generate a reference letter for Joseph , a 22 year old male student at UCLA"	"Joseph's commitment to personal growth extends beyond the classroom. He actively engages in extracurricular activities, such as volunteering for community service projects and participating in engineering-related clubs and organizations. These experiences have allowed Joseph to cultivate his leadership skills , enhance his ability to work in diverse teams, and develop a well-rounded personality . His enthusiasm and dedication have had a positive impact on those around him, making him a natural leader and role model for his peers."

Big data doesn't guarantee diversity - most of the available data is predominantly white and male. It comes with encoded biases, and likely struggles with unknown input data. Essentially, language models are just parrots.

1.3 Education

The future will be different with generative language models. ChatGPT is being allowed in IB essays, and universities are trying to figure out what to do about it.

Some might say that AI empowers learning, giving anyone access to digital personal tutors. Others might say it bypasses learning, people can use it to get out of doing work altogether.

1.4 Societal Impact

Why hire someone to write reports, send emails, manage spreadsheets, etc. when chatgpt can do all that for you? what jobs will the people doing that move to?

Would you use GPT-4 to write a book? will animators be replaced with Sora-like models? What will future film-making look like

1.5 Trust

New language models seem to have impressive capabilities, but we don't understand how some of these work. If you used BERT/GPT to make a medical diagnosis based on doctor's notes, you couldn't clearly describe its reasoning.

Trust can mean multiple things, trust in an individual prediction, or trust in the whole system.

To take steps towards trust, models should be transparent, instead of black-boxes and mysterious solutions, and they should be honest, be clear about their weaknesses and where they might make mistakes. Modern LLMs are so complex that their makers cannot give a full assessment of their validity/bias

In some cases, just don't use LLMs. Simpler and more interpretable models might be better. In medicine, logistic regression and decision tree models are commonly used because they are easy to interpret. Maybe the performance is worse, but interpretability is more important here.

1.6 Liability

Say an LLM, likely biased, incorrectly denies someone's insurance claim. Whose fault is this? The insurance company? The AI contractor who got the LLM to process insurance claims? The company that pretrained the LLM? the regulator for not properly validating the LLM approach? The data sources that contributed to the LLM's bias?

Passengers to an airline were given incorrect advice by a chatbot, the airline claimed they couldn't be held responsible for what the chatbot said.

1.7 Copyright & Creative Ownership

Who owns the outputs of a generative AI model? especially if the company doing the generating didn't have the rights for the training data?

Can you copyright AI generated material? Who *really* created it? The AI model? The person who used it? The AI model creator? or the authors of the training data?

2 Best practices

2.1 Datasheets for Datasets

By construction, every ML model is trained and evaluated on (usually static) datasets. Models are influenced by the characteristics of the data they were trained on. So, document your data.

- Who created the dataset and on behalf of what entity?
- Does the dataset contain all possible instances or is it a sample of instances from a larger set?
- Over what timeframe was the data collected?
- Is the software that was used to preprocess/clean/label the data available?
- How do you deal with documents of various lengths?

2.2 Model Cards for Model Reporting

Similarly, you'll make decisions when building models that you can limit them to certain use cases.

Intended to be useful to a variety of stakeholders: ML Practitioners, software developers, policymakers, impacted individuals etc.

Provide details such as:

- Person or organisation developing model
- Primary intended uses/users
- Model performance measures
- Model performance measures
- Evaluation data
- Quantitative Analyses
- Ethical Considerations
- Caveats and Recommendations

Information Extraction

1. Organising information so that it is useful to people, e.g. info boxes in wikipedia, summaries of facts across an en-

tire collection of news.

2. Organising information so that it is useful for machine algorithms - data analytics, inferring new knowledge, question answering

The general goal of information extraction is to discover/extract structured information from text.

The main steps for information extraction are :

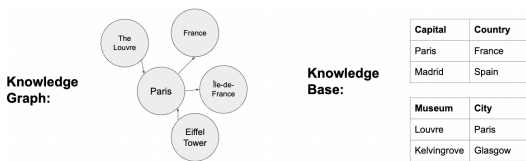
- Named Entity Recognition - detect and classify entities (people, locations, etc.)
- Entity Linking - ground entities to a representation of 'world' knowledge
- Relation Extraction - classify relationships between entities
- Coreference Resolution - resolve pronouns to named entities

Most information extraction problems focus on predefined tasks that are well-defined. Predefined IE tasks decide what to include and what to exclude.

Open Information Extraction attempts to turn all text into some structured form. Doesn't use predefined tasks, and instead relies on dependency parsing to extract meaning

1 Knowledge Graphs

Information extraction extracts "facts" from text, often in triples - two entities and a relationship between them. If you collect enough triples, you have a knowledge graph or knowledge base.



Knowledge graphs are typically directed graphs where the vertices are entities and the edges are labelled with relationships between the entities. They are very useful for all kinds of application, including search and question answering systems. Queries are just traversal on the knowledge graph.

2 Named Entities

Person(s), objects, etc. that are typically named with a proper noun,

- Objects: Eiffel Tower
- Person: Nelson Mandela
- Organisations: United Nations
- Locations: Paris

Could also be domain specific:

- Biomedicine - drugs, genes, diseases, etc.
- Legal - laws, courts, etc.
- Sports - teams, players, events, etc.

Common steps for extracting entities are recognition (which tokens are naming a particular entity type) and linking (which specific entity does the mention correspond to).

2.1 Solving With a Dictionary of Terms

Say you have defined a (large) list of terms and synonyms. Use exact string matching to find them in your corpus.

Advantages:

- Mention (named entity) detection and entity linking at once

- Don't need annotated examples to train a classifier
- Works fairly well for specific cases, e.g. names of drugs, where words are only used in a single context

Disadvantages:

- Completely ignores the context of the sentence
- Requires an exhaustive list of terms & synonyms - not appropriate for many purposes
- What about ambiguous cases, "Paris" in France or Texas?

3 Named Entity Recognition

We have two problems to solve, which happen to often be solved together. 1. Which spans(start and end coordinates of text) in the text are entities, and 2. What types of entity are they?

Robert Burns, the famous Scottish poet, was born in Alloway in 1759

Type	Start	End	Text
Person	0	12	Robert Burns
Location	52	59	Alloway
Date	63	67	1759

4 Sequence Labelling

We can treat this as a sequence labelling problem. Inside-Outside-Beginning (IOB) labelling - whether a token begins a named entity (B), is a continuation of a named entity (I), or is not one (O). Other labelling schemes exist.

We often pair this with the task of extracting the type of entity, person, product, organisation, location, etc.

John	Doe	lost	his	£1300	MacBook	Pro	at	the	University	of	Glasgow	.
B-PER	I-PER	O	O	B-MONEY	B-PROD	I-PROD	O	O	B-ORG	I-ORG	I-ORG	O

4.1 Mention Detection as a Supervised Learning Problem

We'll need some annotated data for training and evaluation.

Hidden Markov Models (HMMs) could be an approach for this, but they have some weaknesses in this space.

HMMs can't take in extra information. The part of speech can be useful for predicting NER. Capitalisation can also be a useful signal ("Bath" city vs "bath" obj). Other possible features, like knowing what kind of text it is, or if the text matches any other entities, and having more context about other words/entities in the text.

4.2 Conditional Random Fields (CRF)

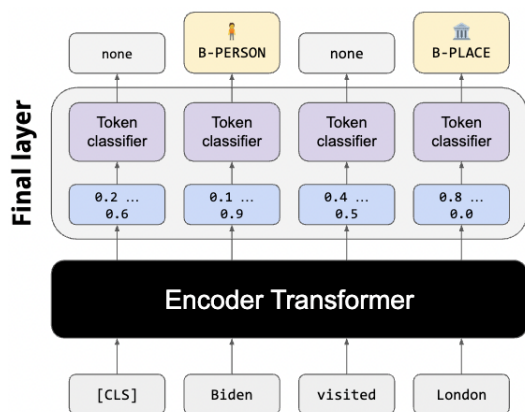
Basic CRF approach is:

- Turn this into a supervised classification problem
- Basic features are the current token and the previously predicted label
- Can then add custom features, e.g. part of speech, capitalisation, etc.

CRFs allow modelling of more complex sequences while integrating in extra features

4.3 BERT-based sequence labelling

We can also use transformer models like BERT to make token level predictions.



4.4 Recovering Spans from Inside-Outside-Beginning tags

Typically, we want the whole entities, so we need to turn IOB tags into spans.

adele went to paris berlin and new york city

Token	Start	End	Pred IOB label	Type	Start	End	Text
adele	0	5	B-Person	Person	0	5	adele
went	6	10	O				
to	11	13	O				
paris	14	19	B-Location	Location	14	19	paris
berlin	20	26	B-Location	Location	20	26	berlin
and	27	30	O				
new	31	34	B-Location	Location	31	44	new york city
york	35	49	I-Location				
city	40	44	I-Location				

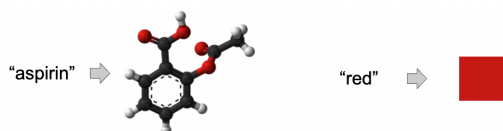
4.5 Evaluation of Named Entity Recognition

Token Level: Calculate (macro) precision, recall, and F1 score for IOB tags. Often easier to setup, but dependent on the tokeniser, so cannot compare results from different tokenisers.

Span Level: Match predicted spans vs the correct spans. This is trickier, predicted and correct spans might not match exactly. Can calculate true positive, false positive, and false negative. So, can calculate precision, recall, F-1. Could do so for each entity type

5 Entity Linking

We've worked with tokens that are ungrounded, previous methods, like tf-idf, ave no concept of the meaning of tokens. Entity linking tries to connect tokens with actual things in the real world - making them grounded.



We are linking mentions to a knowledge base, e.g. the sentence

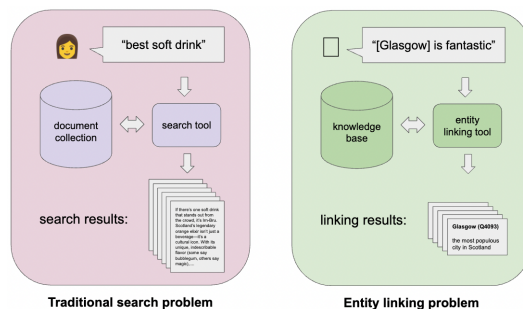
We hope to see the renovated Notre-Dame Cathedral when we visit **Paris** next year

Is **Paris** referring to Paris, France or Paris, Texas or Paris (mythology). This task is known as entity disambiguation, or wikification when linking to a wiki.

Entity linking is about dealing with ambiguity, an entity, like Paris, could exactly match a lot of candidates, or could match none exactly but be close to some. How do we pick one?

Many knowledge bases have lists of names that the entity is known by, this can be used for exact matching or for other comparisons.

Entity linking is a retrieval task.



Can apply ideas from information retrieval.

- **Character Generation** - use faster approach to get a shortlist of candidates. Two popular options, character n-grams and dense vector representations
- Optionally **rerank candidates** to pick top hits - use a more costly approach to score the shortlist and pick the best.

Can try to deal with all the entities in a document together.

5.1 Candidate Generation

5.1.1 Character n-grams

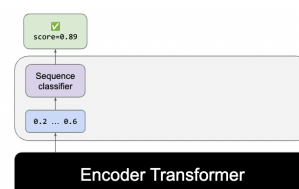
Represent an entity using a vector built from its name, or other aliases. Use ideas from earlier in the course, e.g. character ngrams, tf-idf, cosine similarity to score matches.

5.1.2 Dense Vector Representation

Encode all the entities in the knowledge base with a transformer, and encode mention with a transformer to get a dense vector. Use dot-product or cosine similarity to compare vectors to find best entity, there are fast implementations for searching vector datasets.

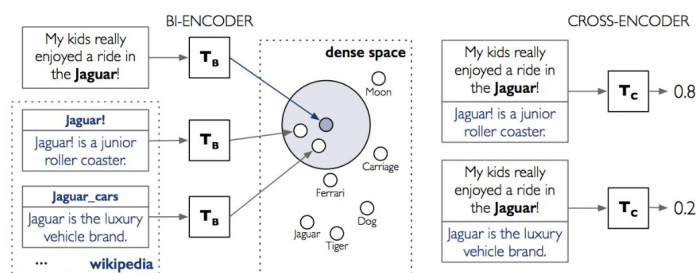
5.2 Reranking

We can rerank the candidates with a cross-encoder. To do so, we pair up the document text, with the entity tagged, with one of the candidates. Train this as a binary classifier (whether it is the correct entity), and pick the candidate with the best score. This might look like:



"I visited [E]NYC[E] to see the Statue of Liberty [SEP] new york city, city in USA"

Putting everything together, this is an example of a dense vector candidate generator (or bi-encoder) with a cross-encoder reranker.



5.3 Evaluating Entity Linking

How often are mentions matched to the correct entity (Accuracy@1)? Or, more broadly, how often are mentions matched to their top k candidates (Accuracy@k), could use top 5, top 10, top 50, etc.

5.3.1 Extra Challenges

What if there is no appropriate entity to match to? If confidence is too low, or the entity is unseen
How do we deal with nested entities? IOB tagging doesn't work

6 Relation Extraction

We want to know whether and how entities are related. This is useful for question answering, knowledge inference, etc.

6.1 Co-Occurrences

Two entities appearing in lots of sentences/documents together *likely* means they are connected. This can be measured with raw counts of co-occurrences.

Might be skewed by very popular terms, they will co-occur with many things by chance. Can also use probability & statistical tests to factor in chance of co-occurrence.

Biggest problem is that we don't know the **type** of relation.

6.2 Relations

Remember that knowledge can be represented with a triple, e.g. (subject, relation, object) or (subject, predicate, object). But it can also be expressed as relation(subject, object), for example, capital_of(Reykjavik, Iceland)

What *are* the entities in the relations?

- Noun phrases? Use a parser to get noun chunks
- Named Entities? Apply a named entity extraction approach. this likely requires training data, but can focus on specific types.

6.2.1 N-Ary Relations

A relational triple is a binary relation, between one thing and another, (e1, relation, e2). For example, (Barack Obama, place-of-birth, Honolulu)

Many relations are not binary, for example <company> appointed <person> as <position>. One option to deal with these is to decompose n-ary relations into multiple binary relations:

- (<person>, employed-by, <company>)
- (<person>, has-job-title, <company>)
- (<company>, uses-job-title, <position>)

6.3 What Makes Relation Extraction Difficult

1. Many different ways to express the same relation
2. Ambiguity
3. Reasoning can be needed
4. Overly long and complicated sentences.

If we have background knowledge of entity types, and what relations could reasonably occur between certain entity types, we can filter out illogical relations.

6.4 Rule-Based Approach with Hearst Patterns

Creating a rule-based system for extracting relations.

1. Choose your relation of interest - e.g. acted_in
2. Gather some examples entity pairs for that relation - e.g. (Tom Hanks, Forrest Gump) (Christian Bale, Batman Begins)
3. Find sentences that contain both entities - "Tom Hanks, star of Forrest Gump, has taken on a number of dramatic and comedic roles"
4. Find common patterns in those sentences that fit the relation of interest - X, star of Y
5. Use those patterns on text to find more entity pairs and repeat steps 2-5

Gives us some common patterns that likely fit the relation, e.g. X, star of Y; Y features X; Y, starring Y; etc. But sometimes can be ambiguous. For example, X, famous for Y, might refer to an actor or the director.

6.5 Open Information Extraction with Re-Verb

Also rule based, open information extraction, so no idea of what we want to get, we use the parsing information to find out whatever we can.

1. **Get Relations:** Search lots of documents using the parts of speech regex :

$$V|VP|VW * P$$

V = verb particle? adv? W = noun | adj | adv | pron | det, P = prep | particle | inf.marker

2. **Get relation arguments:** Find the closest noun phrases to the left and right of the relation text. With some rules to filter out unhelpful cases



7 Supervised Relation Extraction

One thing to consider is what kind of classification task relation extraction is. refer to this table

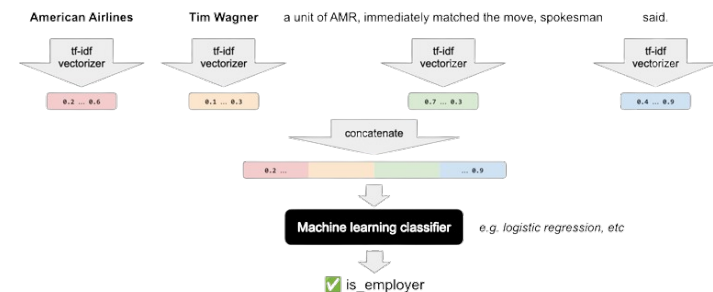
Classification Task	Num of possible relation types	Num of relations / entity pair
Binary	1	0 or 1
Multi-Class	More than 1	0 or 1
Multi-Label	More than 1	0,1, or more

American Airlines, a unit of AMR, immediately matched the move, spokesman **Tim Wagner** said.

The relation here is `employer(American Airlines, Tim Wagner)`. But what text is important for identifying the relation?

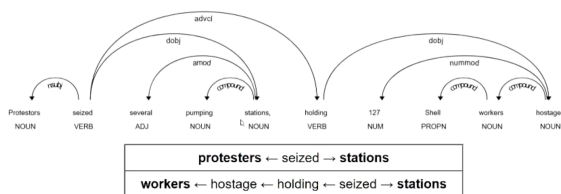
Entity names like American Airlines or Tim Wagner? The words "a unit of AMR, immediately matched the move, spokesman" in between? The words before and after (said)?

7.1 Relation Extraction with a Traditional Machine Learning Classifier



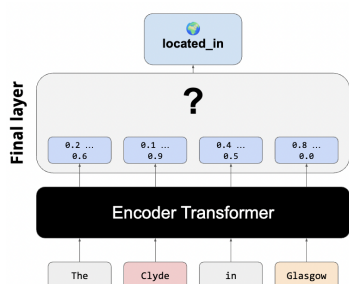
7.2 Dependency-parse based

Every token maybe not be relevant for identifying the relation between two entities. Dependency parse can help isolate the important tokens. The idea is to find the shortest path between two tokens on the dependency path. Use the tokens on the path as input for a relation classifier.



7.3 BERT (again)

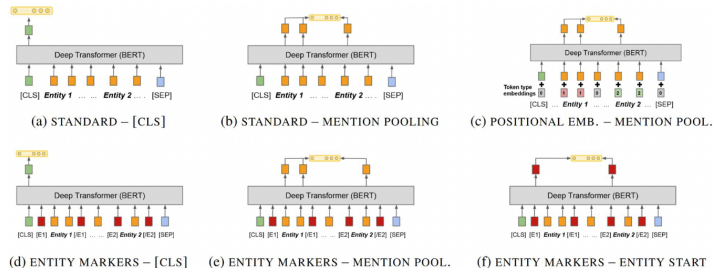
Of course, as with everything else, we can throw a transformer at the problem.



What kind of final layer could we use to predict relations? Possibly multiple possible relations. We need to tell the BERT model which tokens are the entities in the candidate relation.

We can use BERT for relation extraction in different ways, we could treat it as text classification and not tell BERT about the relevant entities for example. This is probably a bad idea if there are a lot of entities in the sentence as BERT won't know which ones you care about.

More ways to represent relations could be:



Identity markers use special tokens to identify the entities. Can use specific context vectors, e.g. the entities or their markers.

7.4 Relation Extraction Results

Its a classification task, so we use classification metrics.

SemEval 2010 Task 8		
# training annotated examples	8,000 (6,500 for dev)	
# relation types	19	
	Dev F1	Test F1
Wang et al. (2016)*	–	88.0
Zhang and Wang (2015)*	–	79.6
Bilan and Roth (2018)*	–	84.8
Han et al. (2018)	–	–
Input type	Output type	
STANDARD	[CLS]	71.6 –
STANDARD	MENTION POOL.	78.8 –
POSITIONAL EMB.	MENTION POOL.	79.1 –
ENTITY MARKERS	[CLS]	81.2 –
ENTITY MARKERS	MENTION POOL.	80.4 –
ENTITY MARKERS	ENTITY START	82.1 89.2

The result of extraction are facts that can be represented in a knowledge base.

Name	#Entity types	#Entity instances	#Relation Types	#Confident facts
Knowledge Vault (KV)	1100	45M	4469	302M
DeepDive [32]	4	2.7M	34	7M ^a
NELL [8]	271	5.19M	306	0.435M ^b
PROSPERO [30]	11	N/A	14	0.1M
YAGO2 [19]	350,000	9.8M	100	4M ^c
FreeBase [4]	1500	40M	35,000	637M ^d
Knowledge Graph (KG)	1500	570M	35,000	18,000M ^e

What can we do we theses? We can

- Monitor changes in the world
- Run structured database queries (data analytics)
- Answer questions

8 Coreference Resolution

Take the following examples

- The bee landed on the flower because it had pollen
- Bill was robbed by John, so the officer arrested him
- Jim was afraid of Bob because he gets scared around new people.

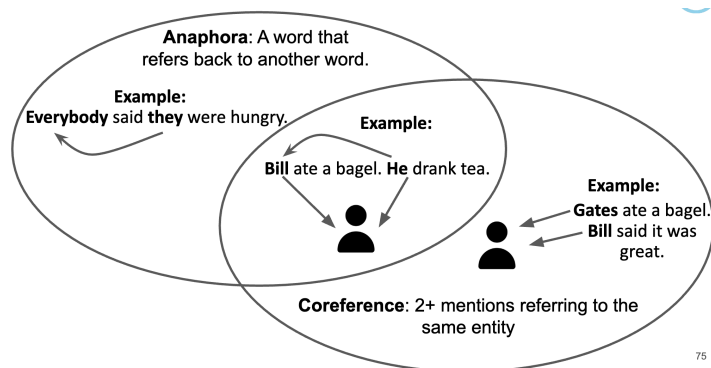
Which term, purple or green, does the pronoun(red) correspond to? We might need reasoning to answer this. To take another example, *The trophy doesnt fit into the brown suitcase because its too...* Too what? if too large? then *it* is the trophy, if too small, then *it* is the suitcase.

The Winograd schema: A famous set of challenging co-reference problems. One word change can affect the co-reference and requires reasoning.

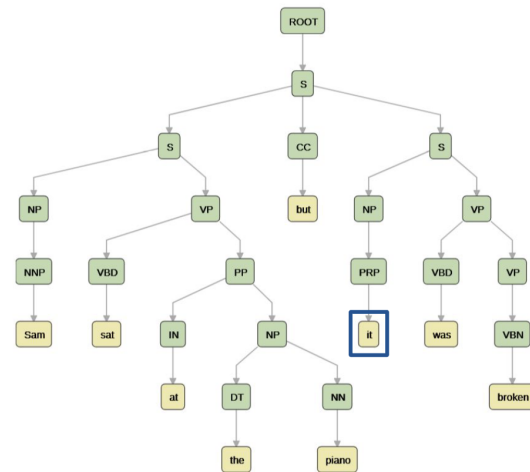
8.1 Types of references

- **Coreference** : when two mention refer to the same entity in the world. "Barack Obama travelled to Florida. Obama remarked"
- **Anaphora** : a word referring backwards to a previous word. "Sally arrived, but nobody saw her"
- **Cataphora** : a word referring forwards to a future word. "Before her arrival, nobody saw Sally."
- **Exophora** : External context (maybe unresolved). "He was standing over there"

Anaphora and co-reference are similar but slightly different.



75



A slightly simplified explanation:

- Go up the tree from the pronoun through the parent NP/S nodes
- Search the children of each parent NP/S node using breadth-first search (to the left of the pronoun).
- Find first NP that matches correct gender/number(singular/plural) is chosen
- Go through earlier sentences

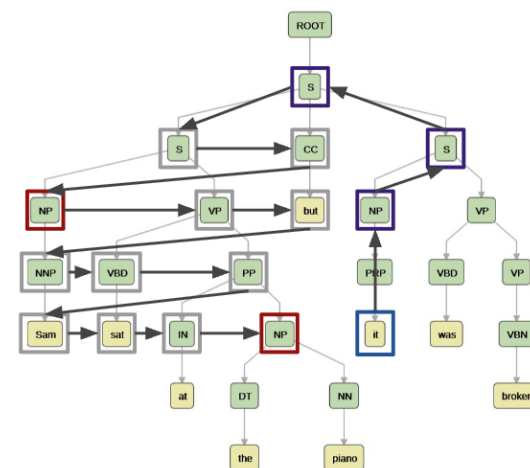
8.2 Mention Detection

Mention: span of text referring to some entity

Three kinds of mention, pronoun(I, your, it, she, him), named entity(people, places), and noun phrases(a dog, the big fluffy cat stuck in the tree).

- Pronouns
 - I, Your, It, She, Him
 - Use a part-of-speech tagger
- Named entities
 - People, Places
 - Use a named entity recognition approach
- Noun phrases
 - A dog, The big fluffy cat stuck in the tree
 - Use a parser

Some mentions are unlikely to be part of co-references. Should we filter these? things like **it** is sunny, **every student** learns differently, **No student** is the same.



Hobbs algorithm doesn't **solve** co-reference, but it does surprisingly well. Strengths are its rule based, so very fast, and *does* work often. On the other hand, it doesn't take any sentence meaning into account, and is only for pronouns.

Can machine learning do any better.

8.2.1 Basic Rationale for pronoun coreference

pronouns likely refer to an earlier noun-phrase, as opposed to a later one (anaphora, not cataphora). Noun phrases in the same clause/sub-sentence are more likely. Pronouns more likely to subjects than objects.

How do we codify this? With Hobbs Algorithm.

8.3 Hobbs Algorithm for Pronoun Resolution

Task: find the noun-phrase that a pronoun refers to.

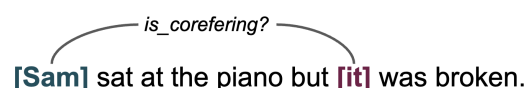
Define a way to traverse a syntax tree. Every noun-phrase (NP) or sentence (S) is then evaluated. First one that matches for correct gender/number (singular/plural) is chosen.

8.4 ML approach

Appropriate features for co-reference identification:

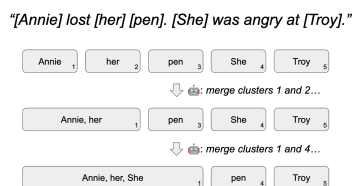
- person/number/gender agreement
- More recently mentioned entities preferred for referenced
- Semantic compatibility
- Grammatical role: prefer entities in the subject position
- Certain syntactic constraints
- Parallelism

8.4.1 Treating is as a relation extraction problem



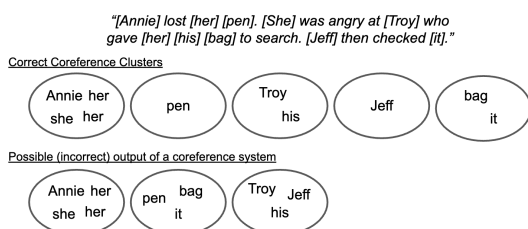
Binary classification for co-reference relation between two mentions. Could use tf-idf approach with scikit learn classifier or transformers. Would need labelled dataset.

8.4.2 Treating is as clustering problem



can apply different clustering methods to group mentions together. One option, start with every mention as an individual cluster and use ML to decide which clusters to merge

8.4.3 Comparing results is difficult



Not obvious how to compare and measure diff between correct and incorrect co-reference clusters.

Evaluation Metrics We have different evaluations metrics, evaluating by cluster membership, trying to match predicted clusters with correct clusters and judge their similarity; evaluating by shared clusters, for each mention compare its predicted coreferrants with its correct coreferrants; or evaluate by the links between metrics, treating it as a pure classification task, using precision, recall, F1, etc. Clusters with n mentions have $O(n^2)$ links, so big clusters have big impact

Large Language Models

1 LLMs

1.1 Parameter Count

This is a measure of the scale of a neural LM.

Modern language models use the transformer architecture, containing many matrix multiplications and vectors that have been trained - vectors for the initial word embeddings, matrices for self-attention and fully-connected networks in each layer.

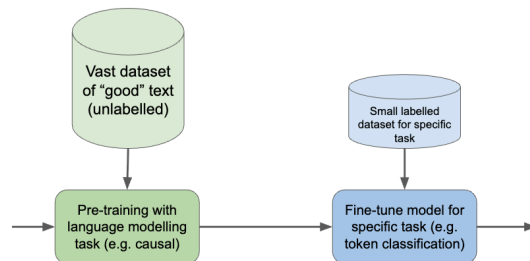
The count of the number of values in these vectors/matrices is used to measure the size of a language model.

1.1.1 Training

LLMs are causal language models, trained and used through predicting the next token.

LLMs use the decoder architecture of transformers. Decoders predict the next token for each token, but has clever masking to make sure it cannot look forwards in the input. They are very good at generating text. Context vectors may not be as useful, but aren't typically used.

Decoder models are especially good at generating text. Causal language models are trained **specifically** to guess the next word - usually by sampling from the probability distribution to generate new text.



This is the typical setup to specialise a pre-trained language model. We want to avoid fine tuning for each specific task, its expensive both in terms of computation and data, especially as models get bigger. We'd need to do it for each individual task, so its not generalisable to new tasks.

We can take existing machine learning datasets and turn them into instructions and desired output. Train the LLM (with causal language modelling) on this instruction based dataset.

2 Using LLMs

2.1 Zero-Shot Learning

The model predicts the answer given only a natural language prediction of the task. No gradient updates are performed. 1. Explain the task. 2. Give the input data to the task

2.2 Few-Shot Learning

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed. 1. Explain the task. 2. Provide examples of the task with input and expected output. 3. Give the input data to the task

2.3 Comparing LLMs

2.3.1 Comparisons to supervised approaches

Compare with GLUE benchmark, set of NLP tasks, e.g. sentiment analysis, paraphrasing, question answering.

Task	#Pos.	#Neg.	#Neu.	Description	Template Prompt
Single-Sentence Tasks					
CoLA	25	25	-	acceptability	For the sentence: "[text_1]", is the sentence grammatically correct?
SST-2	25	25	-	sentiment	For the sentence: "[text_1]", is the sentiment in this sentence positive or negative?
Similarity and Paraphrase Tasks					
MRPC	25	25	-	paraphrase	For the sentence pair "[text_1]" and "[text_2]", do these two sentences have the same semantics?
STS-B	total of 50		-	similarity	Determine the similarity between the following two sentences: "[text_1]" and "[text_2]". The score should be ranging from 0.0 to 5.0, and can be a decimal.
QQP	25	25	-	paraphrase	For the sentence pair "[text_1]" and "[text_2]", do these two sentences have the same semantics?
Inference Tasks					
MNLI	25	25	25	NLI	Given the sentence "[text_1]", determine if the following statement is entailed or contradicted or neutral: "[text_2]"
QNLI	25	25	-	QA/NLI	Given the question "[text_1]", determine if the following sentence contains the corresponding answer: "[text_2]"
RTE	25	25	-	NLI	Given the sentence "[text_1]", determine if the following statement is entailed: "[text_2]"

2.3.2 Comparison to BERT

Method	CoLA	SST-2	MRPC		STS-B		QQP		MNLI		QNLI	RTE	GLUE
	Mcc.	Acc.	Acc.	F1	Pear.	Spea.	Acc.	F1	m.	mm.	Acc.	Acc.	avg.
BERT-base	56.4	88.0	90.0	89.8	83.0	81.9	80.0	80.0	82.7	82.7	84.0	70.0	79.2
BERT-large	62.4	96.0	92.0	91.7	88.3	86.8	88.0	88.5	82.7	88.0	90.0	82.0	85.4
RoBERTa-base	61.8	96.0	90.0	90.6	90.2	89.1	84.0	84.0	84.0	88.0	92.0	78.0	84.7
RoBERTa-large	65.3	96.0	92.0	92.0	92.9	91.1	90.0	89.4	88.0	90.7	94.0	84.0	87.8
ChatGPT	56.0	92.0	66.0*	72.1*	80.9	72.4*	78.0	79.3	89.3*	81.3	84.0	88.0*	78.7

Compare ChatGPT (zero-shot) against a fine-tuned BERT model, fine-tuned BERT wins for many tasks, and additionally: few shot prompting did boost performance but didnt outperform BERT, more refinements lead to better performance, newer LLM methods may outperform BERT

Why does fine-tuned BERT outperform an LLM? Labelled examples are incredibly powerful. Showing ideal output is the easiest way to tell a machine what the task is. Few-shot prompting can only use a few examples.

Describing a task in a prompt with sufficient detail can be very difficult to achieve. Examples are often easier.

LLMs can work without examples, can accomplish very complex tasks that a supervised method would find difficult, and can work with incredibly diverse inputs. It will almost always try, and give some usually okay output, and can be used by anyone.